

10

CAPÍTULO

Programación dinámica

La programación dinámica es una técnica matemática útil para la toma de decisiones secuenciales interrelacionadas. Proporciona un procedimiento sistemático para determinar la combinación óptima de decisiones.

En contraste con la programación lineal, no cuenta con una formulación matemática estándar “del” problema de programación dinámica, sino que se trata de un enfoque de tipo general para solucionar problemas; además, las ecuaciones específicas que se usan deben ajustarse a la situación particular. Por tanto, es necesario cierto grado de creatividad y un buen conocimiento de la estructura general de los problemas de programación dinámica para reconocer cuándo y cómo un problema puede ser resuelto por medio de estos procedimientos. Es posible desarrollar mejor estas habilidades mediante la exposición de una gran variedad de aplicaciones de programación dinámica y con el análisis detallado de las características comunes de todas estas situaciones. Con este fin, se presentará un gran número de ejemplos ilustrativos.

10.1 EJEMPLO PROTOTIPO DE PROGRAMACIÓN DINÁMICA

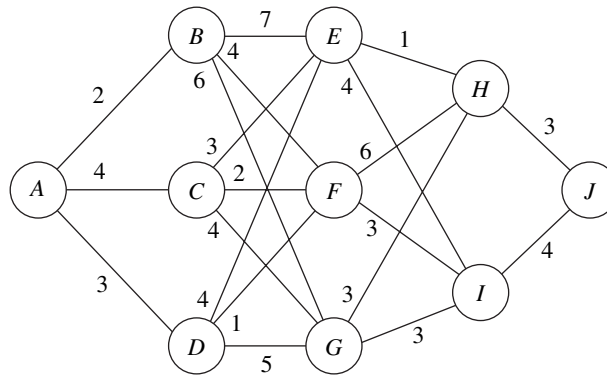
EJEMPLO 1 El problema de la diligencia

EL PROBLEMA DE LA DILIGENCIA se construyó¹ especialmente para ilustrar las características e introducir la terminología de la programación dinámica. Este paradigma se refiere a un cazafortunas mítico de Missouri que decide ir al oeste a sumergirse en la fiebre del oro que surgió en California a mediados del siglo XIX. Tiene que hacer el viaje en diligencia a través de territorios sin ley, donde existen serios peligros de ser atacado por merodeadores. A pesar de que su punto de partida y su destino son fijos, tiene muchas opciones en cuanto a qué estados —o territorios— debe elegir como puntos intermedios. En la figura 10.1 se muestran las rutas posibles, en donde cada estado se representa mediante un círculo con una letra; además, en el diagrama, la dirección del viaje es siempre de izquierda a derecha. Como se puede observar, se requieren cuatro etapas —jornadas en diligencia— para viajar desde su punto de partida en el estado A (Missouri) a su destino en el estado J (California).

Este cazafortunas es un hombre prudente preocupado por su seguridad. Después de reflexionar un poco ideó una manera bastante ingeniosa para determinar la ruta más segura. Se ofrecen pólizas de seguros de vida a los pasajeros. Como el costo de la póliza de cualquier jornada en la diligencia está basado en una evaluación cuidadosa de la seguridad del recorrido, la ruta más segura debe ser aquella cuya póliza represente el menor costo total.

El costo de la póliza estándar del viaje en diligencia, del estado i al estado j , que se denota como c_{ij} , es

¹ Este problema fue desarrollado por el profesor Harvey M. Wagner cuando estaba en Stanford University.



■ **FIGURA 10.1**
Sistema de caminos y
costos del problema de la
diligencia.

	B	C	D		E	F	G		H	I		J
A	2	4	3	B	7	4	6	E	1	4	H	3
				C	3	2	4	F	6	3	I	4
				D	4	1	5	G	3	3		

Estos costos se muestran en la figura 10.1.

La atención se centrará en la pregunta: ¿Cuál es la ruta que minimiza el costo total de la póliza?

Solución del problema

Observe primero que el procedimiento poco inteligente de elegir la ruta más barata en cada etapa sucesiva no conduce a una decisión óptima global. En caso de adoptar esta estrategia, se obtiene la ruta $A \rightarrow B \rightarrow F \rightarrow I \rightarrow J$, con un costo total de 13. Sin embargo, un pequeño sacrificio en una etapa puede permitir mayores ahorros más adelante. Por ejemplo, $A \rightarrow D \rightarrow F$ es, en total, más barato que $A \rightarrow B \rightarrow F$.

Un enfoque posible para resolver este problema es el de prueba y error.² Sin embargo, el número de rutas posibles es grande (18) y el cálculo del costo total de cada ruta no es una tarea atractiva.

Por fortuna, la programación dinámica proporciona una solución con mucho menos esfuerzo que la enumeración exhaustiva. (El ahorro computacional es enorme cuando se trata de versiones más grandes de este problema.) La programación dinámica comienza con una pequeña porción del problema original y encuentra la solución óptima para este problema pequeño. Después agranda de manera gradual el problema y encuentra la solución óptima actual a partir de la que le precede, hasta resolver el problema original completo.

En el caso del problema de la diligencia, se comienza con el problema sencillo que plantea que el cazafortunas casi ha llegado al final de su viaje y sólo tiene una etapa más —una jornada en la diligencia— por recorrer. La solución óptima obvia de este problema reducido es ir del estado actual —el que sea en el que se encuentre— a su destino final (estado J). En cada una de las iteraciones siguientes, el problema aumenta de una en una el número de etapas que le quedan por recorrer para completar el viaje. En cada problema aumentado se puede encontrar la solución óptima del lugar al que debe dirigirse desde cada estado posible, donde se toman en cuenta los resultados obtenidos en la iteración anterior. A continuación se describen los detalles de este procedimiento.

² Este problema también puede formularse como un *problema de la ruta más corta* (vea la sección 9.3), en el cual los *costos* tienen el papel de las *distancias* en el problema de la ruta más corta. En realidad, el algoritmo de la sección 9.3 utiliza la filosofía de programación dinámica. Sin embargo, debido a que el problema en estudio tiene un número fijo de etapas, el enfoque de programación dinámica que se presentó es el mejor.

Formulación. Sean x_n ($n = 1, 2, 3, 4$) las variables de decisión que representan el destino inmediato de la etapa n (el n -ésimo viaje que se hará en diligencia). En este caso, la ruta seleccionada es $A \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$, donde $x_4 = J$.

Sea $f_n(s, x_n)$ el costo total de la mejor *política* global para enfrentar las etapas *restantes*, mientras el agente de ventas se encuentra en el estado s , listo para iniciar la etapa n y elige x_n como destino inmediato. Dados s y n , sea x_n^* el valor de x_n —no necesariamente único— que minimiza $f_n(s, x_n)$, y sea $f_n^*(s)$ el valor mínimo correspondiente de $f_n(s, x_n)$. Entonces,

$$f_n^*(s) = \min_{x_n} f_n(s, x_n) = f_n(s, x_n^*),$$

donde

$$\begin{aligned} f_n(s, x_n) &= \text{costo inmediato (etapa } n) + \text{costo futuro mínimo (etapas } n+1 \text{ en adelante)} \\ &= c_{sx_n} + f_{n+1}^*(x_n). \end{aligned}$$

El valor de c_{sx_n} está dado por las tablas anteriores para c_{ij} al establecer $i = s$ (el estado actual) y $j = x_n$ (el destino inmediato). Como el destino final (estado J) se alcanza al terminar la etapa 4, $f_5^*(J) = 0$.

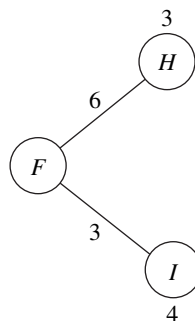
El objetivo es encontrar $f_1^*(A)$ y la ruta correspondiente. La programación dinámica la encuentra al determinar en forma sucesiva $f_4^*(s), f_3^*(s), f_2^*(s)$, para cada uno de los estados posibles s y usar después $f_2^*(s)$ para encontrar $f_1^*(A)$.³

Procedimiento de solución. Cuando el cazafortunas tiene sólo una etapa por recorrer ($n = 4$), su ruta de ahí en adelante está perfectamente determinada por su estado actual s (ya sea H o I), así como su destino final, $x_4 = J$, de manera que la ruta de esta última jornada en diligencia es $s \rightarrow J$. Por tanto, $f_4^*(s) = f_4(s, J) = c_{s,J}$, la solución inmediata al problema para $n = 4$ es

$n = 4$:

s	$f_4^*(s)$	x_4^*
H	3	J
I	4	J

Cuando el cazafortunas tiene dos etapas por recorrer ($n = 3$), el procedimiento de solución requiere unos cuantos cálculos. Por ejemplo, suponga que se encuentra en el estado F . Entonces, como se describe en el diagrama, debe ir al estado H o al estado I con unos costos inmediatos respectivos de $c_{F,H} = 6$ o $c_{F,I} = 3$. Si elige el estado H , el costo adicional mínimo al llegar ahí se presenta en la tabla anterior como $f_4^*(H) = 3$, como se muestra sobre el nodo H del diagrama. En consecuencia, el costo total de esta decisión es $6 + 3 = 9$. Si en su lugar elige el estado I , el costo total es $3 + 4 = 7$, que es menor. Por tanto, la opción óptima es esta última, $x_3^* = I$, puesto que proporciona el costo mínimo, $f_3^*(F) = 7$.

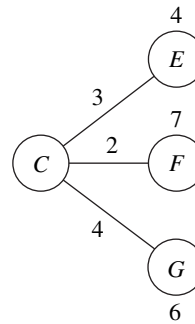


³ Debido a que el procedimiento implica un recorrido *hacia atrás* etapa por etapa, algunos autores cuentan n también hacia atrás para denotar el número de *etapas que faltan* para llegar al destino. Para simplificar, aquí se usa la forma natural de *contar hacia adelante*.

Son necesarios cálculos similares cuando se parte de los otros dos estados posibles $s = E$ y $s = G$ con dos jornadas por delante. Intente obtener la respuesta con la ayuda tanto de un diagrama (figura 10.1) como del álgebra [combine los valores de c_{ij} y $f_4^*(s)$], para verificar los siguientes resultados del problema con $n = 3$.

$n = 3:$	$s \backslash x_3$	$f_3(s, x_3) = c_{sx_3} + f_4^*(x_3)$		$f_3^*(s)$	x_3^*
		H	I		
	E	4	8	4	H
	F	9	7	7	I
	G	6	7	6	H

La solución de la segunda etapa ($n = 2$), cuando quedan tres jornadas por recorrer, se obtiene en forma parecida. En este caso, $f_2(s, x_2) = c_{sx_2} + f_3^*(x_2)$. Por ejemplo, suponga que el cazafortunas se encuentra en el estado C , como se muestra en el siguiente diagrama.



Ahora deberá ir al estado E , F o G con costos inmediatos respectivos de $c_{C,E} = 3$, $c_{C,F} = 2$ o $c_{C,G} = 4$. Al llegar a este punto, el costo adicional mínimo hasta llegar al destino se presenta en la tabla de $n = 3$ como $f_3^*(E) = 4$, $f_3^*(F) = 7$ o $f_3^*(G) = 6$, respectivamente, como muestra el número colocado arriba de los estados E , F y G en el diagrama anterior. A continuación encontrará un resumen con los cálculos que resultan de las tres alternativas.

$$\begin{aligned} x_2 = E: \quad f_2(C, E) &= c_{C,E} + f_3^*(E) = 3 + 4 = 7. \\ x_2 = F: \quad f_2(C, F) &= c_{C,F} + f_3^*(F) = 2 + 7 = 9. \\ x_2 = G: \quad f_2(C, G) &= c_{C,G} + f_3^*(G) = 4 + 6 = 10. \end{aligned}$$

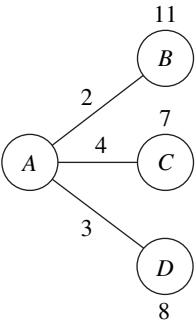
El mínimo de estos tres números es 7, por lo que el costo total mínimo desde el estado C al final es $f_2^*(C) = 7$, y el destino inmediato debe ser $x_2^* = E$.

Al hacer cálculos similares cuando se parte del estado B o D (intente esto) se llega a los siguientes resultados para el problema de $n = 2$:

$n = 2:$	$s \backslash x_2$	$f_2(s, x_2) = c_{sx_2} + f_3^*(x_2)$			$f_2^*(s)$	x_2^*
		E	F	G		
	B	11	11	12	11	E o F
	C	7	9	10	7	E
	D	8	8	11	8	E o F

En el primer y tercer renglones de esta tabla observe que E y F empatan como el valor que minimiza x_2 , de manera que el destino inmediato desde cualquiera de los estados B o D debe ser $x_2^* = E$ o F .

Si se pasa al problema de la primera etapa ($n = 1$), con las cuatro etapas por recorrer, los cálculos son parecidos a los que se acaban de mostrar para la segunda etapa ($n = 2$), excepto que ahora sólo hay *un* inicio posible, $s = A$, como se muestra en el siguiente diagrama.



A continuación se resumen estos cálculos sobre los tres destinos inmediatos posibles:

$$\begin{aligned} x_1 = B: \quad & f_1(A, B) = c_{A,B} + f_2^*(B) = 2 + 11 = 13. \\ x_1 = C: \quad & f_1(A, C) = c_{A,C} + f_2^*(C) = 4 + 7 = 11. \\ x_1 = D: \quad & f_1(A, D) = c_{A,D} + f_2^*(D) = 3 + 8 = 11. \end{aligned}$$

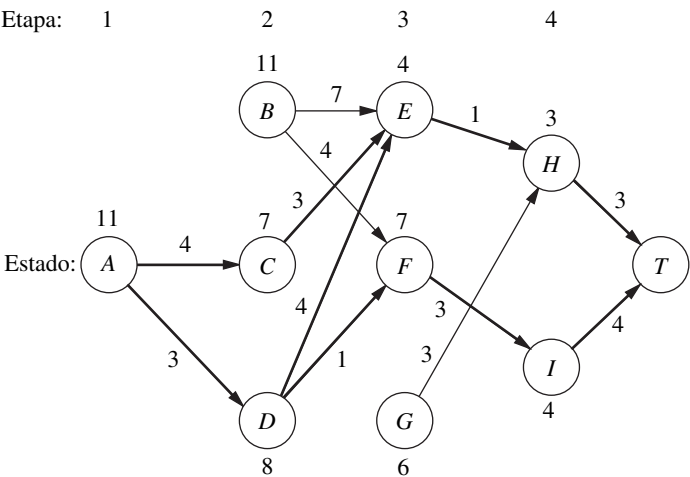
Como el mínimo es 11, $f_1^*(A) = 11$ y $x_1^* = C$ o D , como se muestra en la siguiente tabla:

$n = 1:$	$s \backslash x_1$	$f_1(s, x_1) = c_{sx_1} + f_2^*(x_1)$			$f_1^*(s)$	x_1^*
		B	C	D		
	A	13	11	11	11	C o D

Ahora es posible identificar una solución óptima a partir de las cuatro tablas. Los resultados del problema con $n = 1$ indican que el cazafortunas debe elegir como primer destino inmediato el estado C o el estado D . Suponga que elige $x_1^* = C$. Con $n = 2$, el resultado de $s = C$ es $x_2^* = E$. Esto conduce al problema de $n = 3$, que resulta en $x_3^* = H$ con $s = E$, y el problema con $n = 4$ indica que $x_4^* = J$ con $s = H$. Por tanto, una ruta óptima es $A \rightarrow C \rightarrow E \rightarrow H \rightarrow J$. Si se elige $x_1^* = D$, se obtienen otras dos rutas óptimas $A \rightarrow D \rightarrow E \rightarrow H \rightarrow J$ y $A \rightarrow D \rightarrow F \rightarrow I \rightarrow J$. Todas tienen un costo total de $f_1^*(A) = 11$.

En la figura 10.2 se resumen estos resultados del análisis de programación dinámica. Observe que las dos flechas de la etapa 1 se obtienen de la primera y última columnas de la tabla de $n = 1$ y el costo se encuentra en la penúltima columna. Cada una de las otras flechas (y el costo resultante) se lee en un renglón de cada una de las otras tablas, exactamente de la misma manera.

■ **FIGURA 10.2**
Descripción gráfica de la solución de programación dinámica del problema de la diligencia. Cada flecha muestra una política de decisión óptima (el mejor destino inmediato) desde ese estado, donde el número del lado del estado es el costo de ahí hasta el final. Al seguir las flechas oscuras desde A hasta T se obtienen las tres soluciones óptimas (las rutas que proporcionan el costo mínimo total de 11).



En la siguiente sección se verá que los términos especiales que describen el contexto particular de este problema —*etapa, estado y política*— en realidad son parte de la terminología general de programación dinámica con una interpretación análoga en otros contextos.

10.2 CARACTERÍSTICAS DE LOS PROBLEMAS DE PROGRAMACIÓN DINÁMICA

El problema de la diligencia es un prototipo literal de los problemas de programación dinámica. En realidad, el ejemplo se diseñó así, con el propósito de disponer de una interpretación física literal de la estructura abstracta de estos problemas. Por tanto, una manera de reconocer una situación que se puede formular como un problema de programación dinámica es poder identificar una estructura análoga a la del problema de la diligencia.

A continuación se presentan y estudian estas características básicas que distinguen a los problemas de programación dinámica.

1. El problema se puede dividir en **etapas**, cada una de las cuales requiere de una **política de decisión**.

En el problema de la diligencia se hizo una división literal en cuatro etapas (viajes) que corresponden a las cuatro jornadas en diligencia. La política de decisión en cada etapa fue qué póliza de seguro elegir, esto es, qué destino elegir para la siguiente jornada en diligencia. De manera parecida, otros problemas de programación dinámica requieren tomar una *serie de decisiones interrelacionadas*, cada una de las cuales corresponde a una etapa del problema.

2. Cada etapa tiene cierto número de **estados** asociados con su inicio.

Los estados asociados con cada etapa del problema de la diligencia son los estados (o territorios) en los que el cazafortunas puede encontrarse al iniciar esa jornada específica del viaje. En general, los estados son las distintas *condiciones posibles* en las que se puede encontrar el sistema en cada etapa del problema. El número de estados puede ser finito —como en el problema de la diligencia— o infinito, como en otros ejemplos subsecuentes.

3. El efecto de la política de decisión en cada etapa es *transformar el estado actual en un estado asociado con el inicio de la siguiente etapa*, quizá según una distribución de probabilidad.

La decisión del cazafortunas sobre su siguiente destino lo conduce de su estado actual al siguiente estado en su viaje. Este procedimiento sugiere que los problemas de programación dinámica se pueden interpretar en términos de las *redes* descritas en el capítulo 9. Cada *nodo* corresponde a un *estado*. La red consistiría en columnas de nodos, donde cada *columna* corresponde a una *etapa*, en forma tal que el flujo que sale de un nodo sólo puede ir a un nodo de la siguiente columna a la derecha. El valor asignado a cada rama que conecta dos nodos puede interpretarse algunas veces como la *contribución inmediata* a la función objetivo que se obtiene al tomar esa política de decisión. En la mayor parte de los casos, el objetivo corresponde a encontrar la trayectoria *más corta* o bien la *más larga* a través de la red.

4. El procedimiento de solución está diseñado para encontrar una **política óptima** para manejar el problema completo, es decir, una receta para elaborar la política de decisión óptima para cada etapa en *cada* uno de los estados posibles.

En el problema de la diligencia, el procedimiento de solución se basa en construir una tabla de cada etapa (n) que prescribe la decisión óptima (x_n^*) para cada estado posible (s). Así, además de identificar las tres *soluciones óptimas* (rutas óptimas) del problema completo, los resultados muestran también cómo debe proceder el cazafortunas en caso de que sea desviado a un estado que no se encuentra en la ruta óptima. En cualquier problema, la programación dinámica proporciona este tipo de receta *política* sobre qué hacer en todas las circunstancias posibles (a esto se debe que la decisión real que se toma al llegar a un estado en particular se llama *política* de decisión). Proporcionar esta información adicional, en vez de sólo especificar una solución óptima —secuencia óptima de decisiones—, puede ser muy valioso en muchas situaciones que incluyen el análisis de sensibilidad.

5. Dado el estado actual, una *política óptima para las etapas restantes* es *independiente* de la política adoptada en *etapas anteriores*. Por tanto, la decisión inmediata óptima depende sólo del estado actual y no de cómo se llegó ahí. Éste es el **principio de optimalidad** de la programación dinámica.

Dado el estado en el que se localiza el cazafortunas, la póliza de seguro de vida óptima (y su ruta asociada) desde este lugar en adelante es independiente de cómo llegó ahí. En general, en los problemas de programación dinámica, el conocimiento del estado actual del sistema expresa toda la información sobre su comportamiento anterior, información que es necesaria para determinar la política óptima de ahí en adelante. (Esta propiedad es la *propiedad markoviana* que se presentará en la sección 16.2.) Un problema que carezca de esta propiedad no se puede formular como un problema de programación dinámica.

6. El procedimiento de solución comienza cuando se determina la *política óptima para la última etapa*.

La política óptima para la última etapa prescribe la política óptima de decisión para *cada* estado posible en esa etapa. Es común que la decisión de este problema de una etapa sea trivial, como lo fue en el problema de la diligencia.

7. Se dispone de una **relación recursiva** que identifica la política óptima para la etapa n , dada la política óptima para la etapa $n + 1$.

En el problema de la diligencia, la relación recursiva que se obtuvo es

$$f_n^*(s) = \min_{x_n} \{c_{sx_n} + f_{n+1}^*(x_n)\}.$$

Entonces, para encontrar la *política óptima de decisión* cuando se comienza en el estado s de la etapa n se necesita encontrar el valor que minimice x_n . El costo mínimo correspondiente se obtiene al usar este valor de x_n para después seguir la política óptima cuando el proceso se encuentra en el estado x_n en la etapa $n + 1$.

La forma precisa de la relación recursiva difiere de un problema a otro de programación dinámica, pero se usará una notación análoga a la que se introdujo en la sección anterior, como se resume a continuación:

N = número de etapas.

n = etiqueta de la etapa actual ($n = 1, 2, \dots, N$).

s_n = *estado* actual de la etapa n .

x_n = variable de decisión de la etapa n .

x_n^* = valor óptimo de x_n (dado s_n).

$f_n(s_n, x_n)$ = contribución de los estados $n, n + 1, \dots, N$ a la función objetivo si el sistema se encuentra en el estado s_n en la etapa n , la decisión inmediata es x_n , y en adelante se toman decisiones óptimas.

$$f_n^*(s_n) = f_n(s_n, x_n^*).$$

La relación recursiva siempre tendrá la forma

$$f_n^*(s_n) = \max_{x_n} \{f_n(s_n, x_n)\} \quad \text{o} \quad f_n^*(s_n) = \min_{x_n} \{f_n(s_n, x_n)\},$$

donde $f_n(s_n, x_n)$ se escribe en términos de $s_n, x_n, f_{n+1}^*(s_{n+1})$ y tal vez alguna medida de la contribución inmediata de x_n a la función objetivo. Lo que hace que la expresión de $f_n^*(s_n)$ sea una relación recursiva es la inclusión de $f_{n+1}^*(s_{n+1})$, en el lado derecho, de manera que $f_n^*(s_n)$ está definida en términos de $f_{n+1}^*(s_{n+1})$.

La relación recursiva recurre constantemente a las etapas posteriores a medida que se trabaja hacia atrás una etapa a la vez. Cuando el número de la etapa actual n disminuye su valor en 1, la nueva función $f_n^*(s_n)$ se obtiene mediante el uso de la función $f_{n+1}^*(s_{n+1})$ que se obtuvo en la iteración anterior; después, el proceso se repite cada nueva iteración. Esta propiedad se refuerza en la siguiente (y última) característica de programación dinámica.

8. Cuando se usa esta relación recursiva, el procedimiento de solución comienza al final y se mueve *hacia atrás* etapa por etapa para encontrar cada vez la política óptima para esa etapa hasta que encuentra la política óptima desde la etapa *inicial*. Esta política óptima lleva de inmediato a una solución óptima para el problema completo, es decir, x_1^* para el estado inicial s_1 , después x_2^* para el estado s_2 que resulta, luego x_3^* para el estado s_3 que se obtiene, y así sucesivamente hasta x_N^* para el estado s_N resultante.

Este movimiento hacia atrás se mostró en el problema de la diligencia, en el que se encontró la política óptima, en forma sucesiva, donde cada estado se iniciaba de las etapas respec-

tivas 4, 3, 2 y 1.⁴ Para todos los problemas de programación dinámica, se obtiene una tabla como la siguiente para cada etapa ($n = N, N - 1, \dots, 1$).

s_n \ x_n	$f_n(s_n, x_n)$	$f_n^*(s_n)$	x_n^*

Cuando se obtiene esta tabla de la etapa inicial ($n = 1$), el problema queda resuelto. Como se conoce el estado de la etapa inicial, la primera decisión está especificada por x_1^* en esta tabla. El valor óptimo de las otras variables de decisión queda, a su vez, especificado por las otras tablas según el estado del sistema que se obtiene al tomar la decisión anterior.

10.3 PROGRAMACIÓN DINÁMICA DETERMINÍSTICA

Esta sección profundiza en el enfoque de programación dinámica en los problemas *determinísticos*, en los cuales el *estado* de la siguiente *etapa* está *determinado por completo* por el *estado* y la *política de decisión* de la *etapa actual*. El caso *probabilístico* en el que existe una distribución de probabilidad del valor posible del siguiente estado se analizará en la sección siguiente.

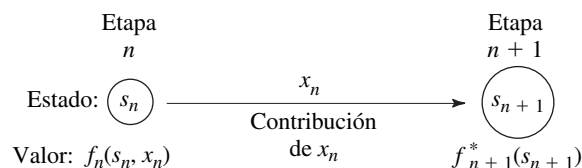
La programación dinámica determinística se puede describir en un diagrama como el de la figura 10.3. En la etapa n el proceso está en algún estado s_n . Al tomar la decisión x_n se mueve a algún estado s_{n+1} en la etapa $n + 1$. La contribución a la función objetivo de ese punto *en adelante* se calculó como $f_{n+1}^*(s_{n+1})$. La política de decisión x_n también contribuye a la función objetivo. Al combinar estas dos cantidades en la forma apropiada se obtiene $f_n(s_n, x_n)$, la contribución de la etapa n en adelante. De la optimización respecto de x_n se obtiene entonces $f_n^*(s_n) = f_n(s_n, x_n^*)$. Una vez determinados x_n^* y $f_n^*(s_n)$ para cada valor posible de s_n , el procedimiento de solución se mueve hacia atrás una etapa.

Una manera de clasificar los problemas de programación dinámica determinística es con base en la *forma de la función objetivo*. Por ejemplo, el objetivo puede ser minimizar la suma de las contribuciones en cada etapa individual —como en el problema de la diligencia—, o maximizar esa suma, o bien minimizar el producto de los términos, etc. Otra clasificación se puede hacer en términos de la naturaleza del *conjunto de estados* en las respectivas etapas. En particular, los estados s_n pueden estar representados por una variable de estado *discreta* —como en el problema de la diligencia—, o por una variable de estado *continua*, o tal vez se requiera un *vector* de estado (más de una variable). De manera similar, las variables de decisión ($x_1, x_2, \dots, x_N$) también pueden ser discretas o continuas.

Se presentarán varios ejemplos para ilustrar estas posibilidades, pero es más importante el hecho de que dichos ejemplos ponen de manifiesto que estas diferencias, en apariencia grandes, en realidad son intrascendentes —excepto en términos de la dificultad de los cálculos— pues la estructura básica de la figura 10.3 permanece igual.

El primer nuevo ejemplo surge en un contexto muy distinto al del problema de la diligencia, pero tiene la misma *formulación matemática*, aunque esta vez se trata de *maximizar* en lugar de minimizar una suma.

■ **FIGURA 10.3**
Estructura básica de
programación dinámica
determinística.



⁴ En realidad, el procedimiento de solución de este problema se puede mover hacia atrás o hacia adelante. Sin embargo, en muchos problemas —en particular cuando las etapas corresponden al *tiempo*—, el procedimiento debe moverse hacia atrás.

Recuadro de aplicación

Seis días después que Saddam Hussein ordenó a sus fuerzas militares iraquíes la invasión de Kuwait el 2 de agosto de 1990, Estados Unidos comenzó un largo proceso consistente en enviar un gran número de unidades militares y cargamento a la región. Después de desarrollar una fuerza de coalición conformada por 35 naciones, se inició la operación militar llamada **Tormenta del Desierto** el 17 de enero de 1991, con el fin de expulsar a las tropas iraquíes de Kuwait. Esta decisión condujo a una rápida victoria por parte de las fuerzas de coalición, que abandonaron Kuwait e invadieron Iraq.

El reto desde el punto de vista logístico involucrado en el transporte expedito de las tropas y cargamento necesarios a la zona de conflicto era enorme. Una misión típica de envío de tropas y carga desde Estados Unidos hasta el golfo Pérsico requería de tres días viaje redondo, tocaba siete o más campos de aviación, consumía alrededor de un millón de libras de combustible y costaba 280 000 dólares. Durante dicha operación, el Comando Militar de Transporte realizó más de 100 de estas misiones diariamente, ya que estaba a cargo de la misión más grande de transporte aéreo de la historia.

Para cumplir con este reto, se aplicó la investigación de operaciones con el fin de desarrollar los sistemas de soporte a las decisiones necesarios para programar y enrutar las misiones de transporte aéreo. La técnica de Investigación de Operaciones que se utilizó para manejar este proceso fue la *programación dinámica*. Las etapas en la formulación de la programación dinámica corresponden a los campos aéreos en

la red de vuelos relevantes para la misión. En un determinado campo aéreo, los estados están caracterizados por el tiempo de salida del y el trabajo disponible que queda de la tripulación actual. La función objetivo que se debía minimizar era la suma ponderada de varias medidas de desempeño: la tardanza en la entrega, el tiempo de vuelo de la misión, el tiempo que se permanece en tierra y el número de cambios de tripulación. Las restricciones incluían una cota inferior en la carga que transportaría la misión y cotas superiores en la disponibilidad de la tripulación y los recursos de soporte en tierra de los campos aéreos.

Esta aplicación de la programación lineal tuvo un efecto dramático respecto a su habilidad para entregar, de manera rápida, la carga y personal necesarios en el golfo Pérsico para apoyar la Operación Tormenta del Desierto. Por ejemplo, cuando se platicó con los desarrolladores de este método, el comandante en jefe de operaciones y transporte de MAC comentó: “Les garantizo que no lo hubiéramos podido hacer (el despliegue de tropas en el golfo Pérsico) sin su ayuda y las contribuciones que ustedes hicieron (los sistemas de soporte a las decisiones); no lo hubiéramos podido hacer en lo absoluto.”

Fuente: M. C. Hilliard, R. S. Solanki, C. Liu, I. K. Busch, G. Harrison y R. D. Kraemer: “Scheduling the Operation Desert Storm Airlift: An advanced Automated Scheduling Support System”, en *Interfaces*, 22(1): 131-146, enero-febrero, 1992.

EJEMPLO 2 Distribución de brigadas médicas entre países

El CONSEJO MUNDIAL DE LA SALUD se dedica a mejorar la atención médica en los países en desarrollo. En la actualidad dispone de cinco brigadas médicas para asignarlas a tres de ellos con el fin de mejorar el cuidado de la salud, la educación para la salud y los programas de capacitación. El consejo debe determinar cuántas brigadas asignar —si lo hace— a cada uno de estos países para maximizar la medida de eficiencia de las cinco brigadas. Éstas deben mantenerse como están constituidas, es decir, el número asignado a cada país debe ser un entero.

La medida de desempeño se evalúa en términos de los *años de vida adicionales por persona*. (En el caso de un país específico, esta medida es igual al *incremento del promedio de vida esperado* en años, multiplicado por su población.) En la tabla 10.1 se proporcionan las estimaciones de estos años de vida adicionales por persona (en múltiplos de mil) para cada país y para cada asignación posible de brigadas médicas.

¿Cuál es la asignación que maximiza la medida de desempeño?

■ **TABLA 10.1** Datos del problema del Consejo Mundial de la Salud

Brigadas médicas	Miles de años-persona de vida adicionales		
	País		
	1	2	3
0	0	0	0
1	45	20	50
2	70	45	70
3	90	75	80
4	105	110	100
5	120	150	130

Formulación. Este problema requiere tomar tres *decisiones interrelacionadas*: cuántas brigadas conviene asignar a cada uno de los tres países. A pesar de que no existe una secuencia fija, estos tres países se pueden considerar como las tres etapas en la formulación de programación dinámica. Las variables de decisión x_n ($n = 1, 2, 3$) son el número de brigadas que se asignan a la etapa (país) n .

La identificación de los estados puede no ser tan evidente. Para determinarlos se hacen preguntas como las siguientes. ¿Qué es lo que cambia de una etapa a la otra? Dado que se han tomado las decisiones en las etapas anteriores, ¿cómo se puede describir el estado de la situación? ¿Qué información sobre el estado actual de las cosas se necesita para determinar la política óptima de aquí en adelante? Con esta base, una opción apropiada para “el estado del sistema” es

$$s_n = \text{número de brigadas médicas todavía disponibles para ser asignadas a los países restantes } (n, \dots, 3).$$

Así, en la etapa 1 (país 1), cuando todavía quedan por asignar brigadas a los tres países, $s_1 = 5$. Sin embargo, en las etapas 2 o 3 (países 2 o 3), s_n es exactamente 5 menos el número de brigadas asignadas en etapas anteriores, de manera que la secuencia de estados es

$$s_1 = 5, \quad s_2 = 5 - x_1, \quad s_3 = s_2 - x_2.$$

Debido al procedimiento de programación dinámica que resuelve hacia atrás etapa por etapa, cuando se trabaja en la etapa 2 o 3 todavía no se han obtenido las asignaciones de las etapas anteriores. Por tanto, se deben considerar todos los estados posibles al iniciar la etapa 2 o 3, es decir, $s_n = 0, 1, 2, 3, 4$ o 5 .

En la figura 10.4 se muestran los estados que deben considerarse en cada etapa. Las ligaduras (segmentos de recta) indican las transiciones posibles de estados de una etapa a la siguiente después de hacer una asignación factible de brigadas médicas al país en cuestión. Los números al lado de las ligaduras son las contribuciones correspondientes a la medida de desempeño, los cuales se tomaron de la tabla 10.1. Desde la perspectiva de esta figura, el problema global es encontrar la trayectoria del estado inicial 5 (inicio de la etapa 1) al estado final 0 (después de la etapa 3) que maximice la suma de los números a lo largo de la ruta.

Para establecer el problema completo en forma matemática, sea $p_i(x_i)$ la medida de desempeño que se obtiene si se asignan x_i brigadas médicas al país i , según los datos de la tabla 10.1. Entonces, el objetivo es elegir x_1, x_2 y x_3 para

$$\text{Maximizar } \sum_{i=1}^3 p_i(x_i),$$

sujeta a

$$\sum_{i=1}^3 x_i = 5,$$

y

las x_i son enteros no negativos.

Si se usa la notación que se presentó en la sección 10.2, se observa que $f_n(s_n, x_n)$ es

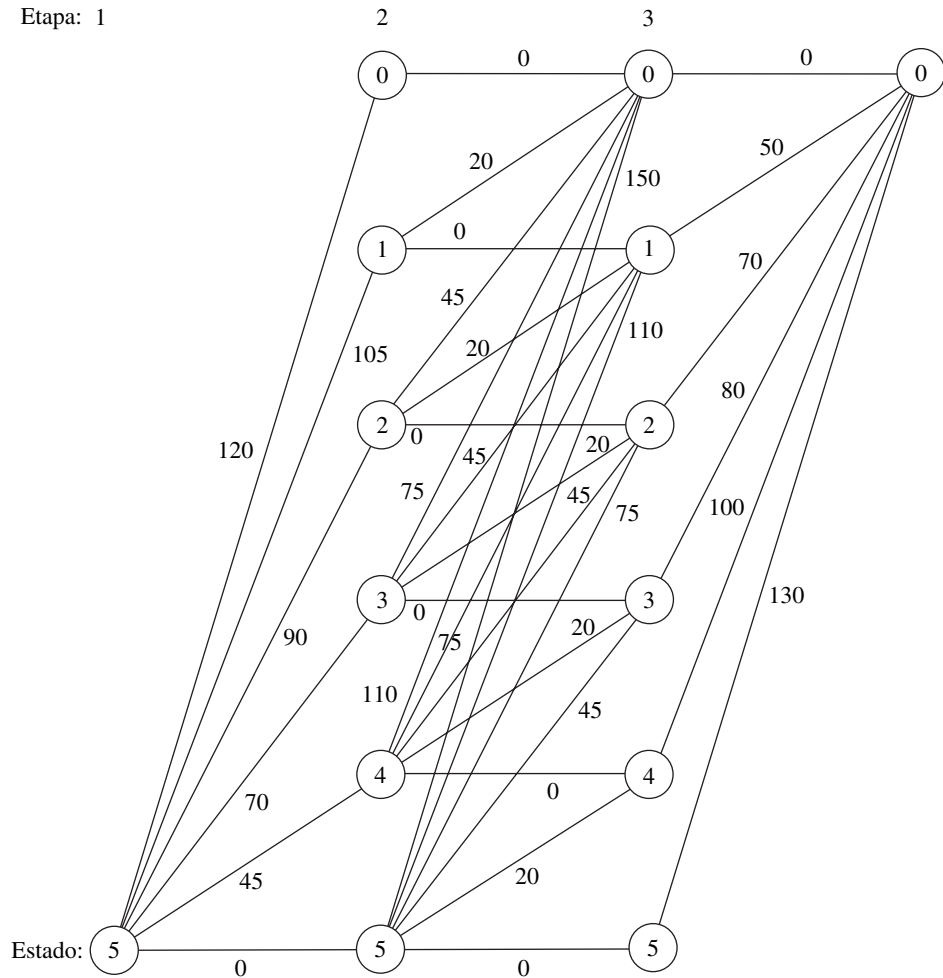
$$f_n(s_n, x_n) = p_n(x_n) + \max \sum_{i=n+1}^3 p_i(x_i),$$

donde el máximo se toma sobre las $x_{n+1}, \dots, x_3$ tales que

$$\sum_{i=n}^3 x_i = s_n$$

y las x_i son enteros no negativos, para $n = 1, 2, 3$. Además,

$$f_n^*(s_n) = \max_{x_n=0,1,\dots,s_n} f_n(s_n, x_n)$$



■ FIGURA 10.4

Descripción gráfica del problema del Consejo Mundial de la Salud que muestra los estados posibles en cada etapa, las transiciones posibles de los estados y las contribuciones correspondientes a la medida de desempeño.

Por lo tanto,

$$f_n(s_n, x_n) = p_n(x_n) + f_{n+1}^*(s_n - x_n)$$

(con f_4^* definido como cero). En la figura 10.5 se resumen estas relaciones básicas.

En consecuencia, la *relación recursiva* que enlaza las funciones f_1^* , f_2^* y f_3^* en este problema es

$$f_n^*(s_n) = \max_{x_n=0,1,\dots,s_n} \{p_n(x_n) + f_{n+1}^*(s_n - x_n)\}, \quad \text{para } n = 1, 2.$$

En el caso de la última etapa ($n = 3$),

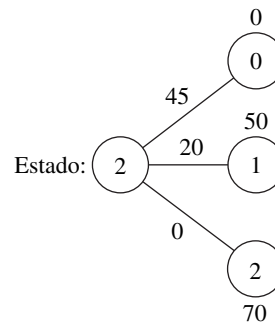
$$f_3^*(s_3) = \max_{x_3=0,1,\dots,s_3} p_3(x_3).$$

A continuación se presentan los cálculos que resultan de la programación dinámica.

Procedimiento de solución. A partir de la última etapa ($n = 3$), se observa que los valores de $p_3(x_3)$, incluidos en la última columna de la tabla 10.1, aumentan hacia abajo de la columna. Entonces, si se dispone de s_3 brigadas médicas para asignar al país 3, el máximo de $p_3(x_3)$ se logra de manera automática cuando se asignan todas las s_3 brigadas; así, $x_3^* = s_3$ y $f_3^*(s_3) = p_3(s_3)$, como se puede ver en la siguiente tabla.

$n = 3$:	s_3	$f_3^*(s_3)$	x_3^*
	0	0	0
	1	50	1
	2	70	2
	3	80	3
	4	100	4
	5	130	5

Ahora el proceso se mueve hacia atrás para comenzar con la penúltima etapa ($n = 2$). Con objeto de determinar x_2^* se necesita calcular y comparar $f_2(s_2, x_2)$ para los distintos valores posibles de x_2 , esto es, $x_2 = 0, 1, \dots, s_2$. La siguiente gráfica ilustra esta situación cuando $s_2 = 2$.



Este diagrama corresponde a la figura 10.5, salvo que ahora se muestran los tres estados posibles en la etapa 3. Así, si $x_2 = 0$, el estado que resulta de la etapa 3 será $s_2 - x_2 = 2 - 0 = 2$, mientras que si $x_2 = 1$ se llega al estado 1 y $x_2 = 2$ conduce al estado 0. Los valores correspondientes de $p_2(x_2)$ en la columna del país 2 de la tabla 10.1 se muestran junto a las ligas y los valores de $f_3^*(s_2 - x_2)$ que provienen de la tabla para $n = 3$ y que se dan junto a los nodos de la etapa 3. Los cálculos requeridos para el caso de $s_2 = 2$ se resumen abajo:

Fórmula: $f_2(2, x_2) = p_2(x_2) + f_3^*(2 - x_2)$.
 $p_2(x_2)$ se localiza en la columna del país 2 de la tabla 10.1
 $f_3^*(2 - x_2)$ es tomado de $n = 3$ de la tabla anterior.

$$x_2 = 0: \quad f_2(2, 0) = p_2(0) + f_3^*(2) = 0 + 70 = 70.$$

$$x_2 = 1: \quad f_2(2, 1) = p_2(1) + f_3^*(1) = 20 + 50 = 70.$$

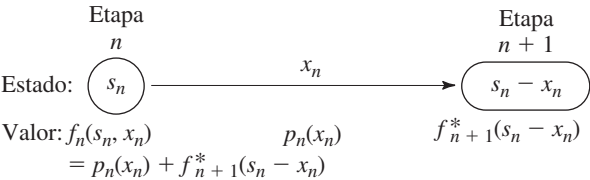
$$x_2 = 2: \quad f_2(2, 2) = p_2(2) + f_3^*(0) = 45 + 0 = 45.$$

Dado que el objetivo es la *maximización*, $x_2^* = 0$ o 1 con $f_2^*(2) = 70$.

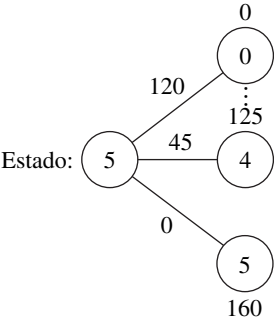
Si se continúa de una manera similar con los otros valores posibles de s_2 (inténtelo) se llega a la siguiente tabla.

$n = 2$:	s_2	x_2	$f_2(s_2, x_2) = p_2(x_2) + f_3^*(s_2 - x_2)$					$f_2^*(s_2)$	x_2^*
			0	1	2	3	4		
	0		0					0	0
	1		50	20				50	0
	2		70	70	45			70	0 o 1
	3		80	90	95	75		95	2
	4		100	100	115	125	110	125	3
	5		130	120	125	145	160	160	4

■ FIGURA 10.5 Estructura básica del problema del Consejo Mundial de la Salud.



En este punto, el procedimiento se mueve hacia atrás para resolver el problema original desde el principio, la etapa 1 ($n = 1$). En este caso, el único estado que debe considerarse es el inicial con $s_1 = 5$, como se describe en seguida.



Si se asignan x_1 brigadas médicas al país 1 se llega al estado $5 - x_1$ brigadas disponibles en la etapa 2, con la elección $x_1 = 0$ se llega al nodo inferior de la derecha, $x_1 = 1$ conduce al siguiente nodo hacia arriba, etc., hasta el nodo superior con $x_1 = 5$. Junto a las ligaduras se muestran los valores correspondientes a $p_1(x_1)$ de la tabla 10.1. Los números junto a los nodos se obtienen de la columna de $f_2^*(s_2)$ de la tabla con $n = 2$. Como antes, se resumen los cálculos de cada valor posible de la variable de decisión que implican sumar los valores de la ligadura correspondiente y el valor del nodo:

Fórmula: $f_1(5, x_1) = p_1(x_1) + f_2^*(5 - x_1)$.

$p_1(x_1)$ se proporciona en la columna del país 1 de la tabla 10.1.

$f_2^*(5 - x_1)$ se presenta en la tabla de $n = 2$.

$x_1 = 0$: $f_1(5, 0) = p_1(0) + f_2^*(5) = 0 + 160 = 160$.

$x_1 = 1$: $f_1(5, 1) = p_1(1) + f_2^*(4) = 45 + 125 = 170$.

$\vdots$

$x_1 = 5$: $f_1(5, 5) = p_1(5) + f_2^*(0) = 120 + 0 = 120$.

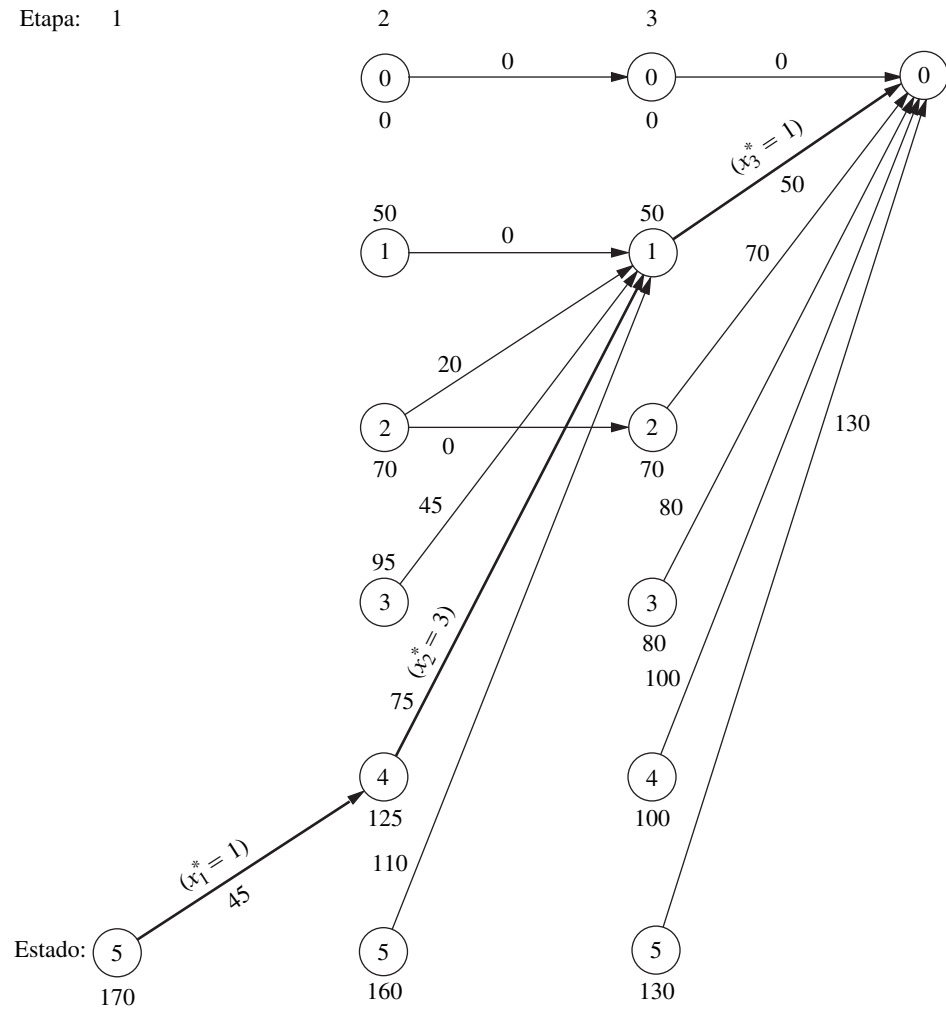
En la siguiente tabla se muestran los cálculos similares para $x_1 = 2, 3, 4$ (inténtelos) que verifican que $x_1^* = 1$ con $f_1^*(5) = 170$.

$n = 1$: $s_1 \backslash x_2$		$f_1(s_1, x_1) = p_1(x_1) + f_2^*(s_1 - x_1)$					$f_1^*(s_1)$	x_1^*
		0	1	2	3	4		
5		160	170	165	160	155	120	1

Así, la solución óptima se obtiene con $x_1^* = 1$, lo que hace que $s_2 = 5 - 1 = 4$, entonces $x_2^* = 3$, lo que hace $s_3 = 4 - 3 = 1$, por tanto $x_3^* = 1$. Como $f_1^*(5) = 170$, esta asignación de (1, 3, 1) brigadas médicas a los tres países conducirá a un total estimado de 170 000 años de vida adicionales, lo que significa por lo menos 5 000 más que para cualquier otra asignación.

Estos resultados del análisis de programación dinámica también se resumen en la figura 10.6.

Etapa: 1



■ FIGURA 10.6

Descripción gráfica de la solución de programación dinámica del problema del Consejo Mundial de la Salud. Una flecha del estado s_n al estado s_{n+1} indica que una política de decisión óptima desde el estado s_n es asignar $(s_n - s_{n+1})$ brigadas médicas al país n . Al asignar las brigadas de esta manera, si se siguen las flechas oscuras desde el estado inicial, se obtiene la solución óptima.

Un problema de tipo prevalente: problema de distribución de esfuerzo

En el ejemplo anterior se ilustra un tipo bastante común de problemas de programación dinámica llamado *problema de distribución de esfuerzo*. En este grupo de problemas existe sólo una clase de *recurso* que debe asignarse a cierto número de *actividades*. El objetivo es determinar cómo distribuir el esfuerzo (el recurso) entre las actividades de la manera más eficaz. En el caso del Consejo Mundial de la Salud, el recurso de que se trata es el conjunto de brigadas médicas, y las tres actividades son los trabajos sobre el cuidado de la salud en los tres países.

Supuestos. Esta interpretación de distribución de recursos entre actividades debe ser familiar al lector ya que es la más común entre los problemas de programación lineal que se presentaron al principio del capítulo 3. Sin embargo, existen algunas diferencias importantes entre el problema de distribución del esfuerzo y la programación lineal que deben ayudar a aclarar en qué difiere la programación dinámica de otras áreas de programación matemática.

Una diferencia clave es que el problema de distribución del esfuerzo incluye sólo *un recurso* (una restricción funcional), mientras que la programación lineal puede manejar cientos o miles de recursos. (En principio, la programación dinámica puede manejar algo más que un recurso, como se verá en el ejemplo 5, en el que se resuelve el problema de tres recursos de la Wyndor Glass Co., pero pronto comienza a perder eficiencia a medida que aumenta el número de recursos.)

Por otro lado, el problema de distribución del esfuerzo es mucho más general que la programación lineal. Considere los cuatro supuestos de programación lineal que se presentaron en la sec-

ción 3.3: proporcionalidad, aditividad, divisibilidad y certidumbre. La *proporcionalidad* se viola por rutina en casi todos los problemas de programación dinámica, incluso los de distribución de esfuerzo (por ejemplo, la tabla 10.1 viola la proporcionalidad). La *divisibilidad* también se viola con frecuencia, como en el ejemplo 2, en donde las variables de decisión deben ser enteras. En realidad, los cálculos de programación dinámica incrementan su grado de complejidad cuando se cumple la divisibilidad (como en los ejemplos 4 y 5). Aunque en este libro se considerará el problema de distribución de esfuerzo sólo bajo condiciones de *certidumbre*, ésta no es necesaria y muchos otros problemas de programación dinámica tampoco respetan este supuesto (como se describe en la sección 10.4).

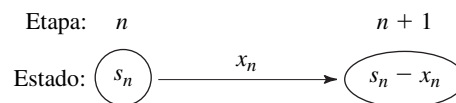
De los cuatro supuestos de programación lineal, el *único* necesario en el problema de distribución de esfuerzo —o en otros problemas de programación dinámica— es el de *aditividad* (o su análogo para funciones que implican el *producto* de términos). Este supuesto se requiere para satisfacer el *principio de optimalidad* de la programación dinámica (característica 5 de la sección 10.2).

Formulación. Debido a que los problemas de distribución de esfuerzo siempre incluyen la asignación de un tipo de recurso a cierto número de actividades, siempre tienen la siguiente formulación de programación dinámica, donde el orden de las actividades es arbitrario:

Etapa n = actividad n ($n = 1, 2, \dots, N$).
 x_n = cantidad de recursos asignados a la actividad n .
 Estado s_n = cantidad de recursos todavía disponibles para asignarse a las actividades restantes ($n, \dots, N$).

La razón para definir el estado s_n de esta manera es que la cantidad de recursos todavía disponibles para asignar es precisamente la información necesaria sobre el estado actual de la situación —al comenzar la etapa n — para tomar las decisiones de asignación para el resto de las actividades.

Cuando el sistema inicia la etapa n en el estado s_n , la elección de x_n siempre da como resultado que el siguiente estado de la etapa $n + 1$ sea $s_{n+1} = s_n - x_n$, como lo muestra el siguiente diagrama:⁵



Observe que la estructura de este diagrama corresponde a la que se muestra en la figura 10.5 en el caso del ejemplo del Consejo Mundial de la Salud de distribución de esfuerzo. Lo que puede diferir de un ejemplo a otro es el *resto* de lo que se muestra en la figura 10.5, es decir, la relación entre $f_n(s_n, x_n)$ y $f_{n+1}^*(s_n - x_n)$, y después la *relación recursiva* que se establece entre las funciones f_n^* y f_{n+1}^* . Estas relaciones dependen de la función objetivo específica del problema.

La estructura del siguiente ejemplo es similar a la del problema del Consejo Mundial de la Salud porque también se trata de un problema de distribución de esfuerzo. Sin embargo, su relación recursiva difiere en que su objetivo es minimizar un producto de términos de las etapas respectivas.

A primera vista, puede parecer que este ejemplo *no* es un problema de programación dinámica determinística pues trata con probabilidades. No obstante, se ajusta a la definición porque en la siguiente etapa el estado queda completamente determinado por el estado y la política de decisión de la etapa actual.

EJEMPLO 3 Distribución de científicos entre grupos de investigación

El grupo encargado de un proyecto espacial del gobierno lleva a cabo una investigación sobre cierto problema de ingeniería que debe resolverse antes de que los seres humanos puedan viajar seguros a Marte. Por el momento, tres equipos de investigación tratan de resolver el problema desde tres puntos de vista diferentes. Se estima que en las circunstancias actuales, la probabilidad de que los

⁵ Esta afirmación supone que x_n y s_n están expresados en las mismas unidades. Si es más conveniente definir x_n como si la cantidad del recurso asignado a la actividad n fuese $a_n x_n$, entonces $s_{n+1} = s_n - a_n x_n$.

respectivos equipos —llamados 1, 2 y 3— fracasen, es de 0.40, 0.60 y 0.80. En consecuencia, la probabilidad actual de que los tres equipos fracasen es $(0.40)(0.60)(0.80) = 0.192$. Como el objetivo es minimizar la probabilidad de fracaso, se asignan al proyecto dos científicos de más alto nivel.

En la tabla 10.2 se proporciona la probabilidad estimada de que los equipos respectivos fracasen si se les asigna 0, 1 o 2 científicos para que colaboren con ellos. Sólo se consideran números enteros de científicos puesto que cada uno de ellos debe dedicar su atención completa a un equipo. El problema es determinar cómo deben asignarse los dos científicos adicionales para minimizar la probabilidad de que los tres equipos fracasen.

Formulación. En razón de que tanto el ejemplo 2 como el 3 son problemas de distribución de esfuerzo, su estructura básica es muy semejante. En este caso, los científicos sustituyen a las brigadas médicas como el recurso con que se cuenta y los equipos de investigación sustituyen a los países que juegan el papel de actividades. Entonces, en lugar de brigadas médicas que se destinan a países, se asignarán científicos a equipos de investigación. La única diferencia básica entre los dos problemas está en su función objetivo.

Debido a que el número de científicos y equipos es tan pequeño, este problema se puede resolver con facilidad mediante un proceso de enumeración exhaustiva. El procedimiento de solución por programación dinámica se presenta con propósitos explicativos.

En este caso, la etapa n ($n = 1, 2, 3$) corresponde al equipo de investigación n , mientras que el estado s_n es el número de científicos *todavía disponible* que debe asignarse a los equipos restantes. Las variables de decisión x_n ($n = 1, 2, 3$) son el número de científicos adicionales que se asigna al equipo n .

Sea $p_i(x_i)$ la probabilidad de fracaso del equipo i si se le asignan x_i científicos adicionales, como se ve en la tabla 10.2. Si el signo Π denota multiplicación, el objetivo del gobierno será elegir x_1, x_2, x_3 para

$$\text{Minimizar } \prod_{i=1}^3 p_i(x_i) = p_1(x_1)p_2(x_2)p_3(x_3),$$

sujeta a

$$\sum_{i=1}^3 x_i = 2$$

y

las x_i son enteros no negativos.

En consecuencia, $f_n(s_n, x_n)$ en este problema es

$$f_n(s_n, x_n) = p_n(x_n) \cdot \min \prod_{i=n+1}^3 p_i(x_i),$$

donde el mínimo se toma sobre $x_{n+1}, \dots, x_3$, tal que

$$\sum_{i=n}^3 x_i = s_n$$

■ **TABLA 10.2** Datos del problema del proyecto espacial del gobierno

Nuevos científicos	Probabilidades de fracaso		
	Equipo		
	1	2	3
0	0.40	0.60	0.80
1	0.20	0.40	0.50
2	0.15	0.20	0.30

y

las x_i son enteros no negativos,para $n = 1, 2, 3$. Así,

$$f_n^*(s_n) = \min_{x_n=0,1,\dots,s_n} f_n(s_n, x_n),$$

donde

$$f_n(s_n, x_n) = p_n(x_n) \cdot f_{n+1}^*(s_n - x_n)$$

(con f_4^* igual a 1). En la figura 10.7 se resumen estas relaciones básicas.Por tanto, en este caso, la *relación recursiva* entre las funciones f_1^*, f_2^* y f_3^* es

$$f_n^*(s_n) = \min_{x_n=0,1,\dots,s_n} \{p_n(x_n) \cdot f_{n+1}^*(s_n - x_n)\}, \quad \text{para } n = 1, 2,$$

y cuando $n = 3$,

$$f_3^*(s_3) = \min_{x_3=0,1,\dots,s_3} p_3(x_3).$$

Procedimientos de solución. Los cálculos de programación dinámica que se obtienen son los siguientes:

$n = 3$:	s_3	$f_3^*(s_3)$	x_3^*
	0	0.80	0
	1	0.50	1
	2	0.30	2

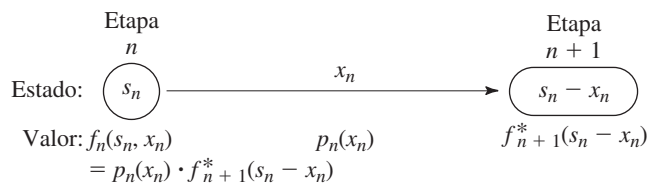
$n = 2$:	$s_2 \backslash x_2$	$f_2(s_2, x_2) = p_2(x_2) \cdot f_3^*(s_2 - x_2)$			$f_2^*(s_2)$	x_2^*
		0	1	2		
	0	0.48			0.48	0
	1	0.30	0.32		0.30	0
	2	0.18	0.20	0.16	0.16	2

$n = 1$:	$s_1 \backslash x_1$	$f_1(s_1, x_1) = p_1(x_1) \cdot f_2^*(s_1 - x_1)$			$f_1^*(s_1)$	x_1^*
		0	1	2		
	2	0.064	0.060	0.072	0.060	1

Por tanto, la solución óptima debe tener $x_1^* = 1$, lo que hace $s_2 = 2 - 1 = 1$, y con esto $x_2^* = 0$, con lo que $s_3 = 1 - 0 = 1$ y $x_3^* = 1$. De esta manera, los equipos 1 y 3 deben recibir un científico adicional cada uno. La nueva probabilidad de que los tres equipos fracasen es de 0.060.

■ FIGURA 10.7

Estructura básica del problema del proyecto espacial del gobierno.



Hasta ahora, todos los ejemplos han presentado variables de estado s_n discretas en cada etapa. Aún más, todos han sido *reversibles* en el sentido de que, de hecho, el procedimiento de solución se puede mover hacia atrás o bien hacia adelante etapa por etapa. (Esta última alternativa se reduce a reenumerar las etapas en el orden inverso y después aplicar el procedimiento estándar.) Esta propiedad de reversibilidad es una característica general de los problemas de distribución de esfuerzo, como los ejemplos 2 y 3, en razón de que las actividades (etapas) se pueden ordenar de cualquier manera.

El siguiente ejemplo es diferente en ambos aspectos; en lugar de estar restringida a valores enteros, la variable de estado s_n en la etapa n es una variable *continua* y puede tomar *cualquier* valor dentro de ciertos intervalos. Como s_n ahora tiene un número infinito de valores, ya no es posible considerar cada uno de sus valores posibles en forma individual. Ahora la solución para $f_n^*(s_n)$ y el valor x_n^* deben expresarse como *funciones* de s_n . Aún más, este ejemplo *no* es reversible porque sus etapas corresponden a *periodos temporales*, por lo que el procedimiento de solución debe ir hacia atrás.

Antes de proceder de manera directa con el siguiente ejemplo, al lector le puede resultar útil revisar los **dos ejemplos adicionales** de programación dinámica determinística que se presentan en la sección Worked Examples del sitio en internet de este libro. El primero se refiere a la planeación de producción e inventarios en cierto número de periodos temporales. Como en los ejemplos que se han estudiado hasta ahora, tanto la variable de estado como la variable de decisión en cada etapa son discretas. Sin embargo, este ejemplo no es reversible porque las etapas corresponden a periodos en el tiempo. Tampoco es un problema de distribución de esfuerzo. El segundo ejemplo es un problema de programación no lineal con dos variables y una sola restricción. Por tanto, aunque es reversible, sus variables de estado y de decisión son continuas. Sin embargo, a diferencia del ejemplo siguiente —que tiene cuatro variables continuas y, por ende, cuatro etapas—, éste tiene sólo dos etapas, por lo que se puede resolver relativamente rápido con programación dinámica y un poco de cálculo.

EJEMPLO 4 Programación del nivel de empleados

La carga de trabajo del LOCAL JOB SHOP está sujeta a grandes fluctuaciones que dependen de la temporada. Sin embargo, es difícil contratar y costoso capacitar a los operadores de las máquinas, por lo que el administrador rechaza la idea de despedir trabajadores durante las temporadas bajas. Tampoco quiere mantener su nómina de temporadas altas cuando no se requiere. Más aún, en forma definitiva se opone a pagar tiempo extra en forma regular. Como todos los trabajos se hacen sobre pedido, no es posible acumular un inventario durante las temporadas bajas. Por tanto, la administración tiene que determinar cuál debe ser la política sobre los niveles de empleados.

A continuación se proporcionan las estimaciones sobre la mano de obra que se requerirá durante las cuatro temporadas del año en un futuro cercano:

Temporada	Primavera	Verano	Otoño	Invierno	Primavera
Requerimientos	255	220	240	200	255

No se permitirá que el nivel de empleados baje de estos niveles. Cualquier contratación más alta se desperdicia con un costo aproximado de \$2 000 por persona por temporada. Se estima que los costos de contratación y despido son tales que el costo de cambiar el nivel de empleados de una temporada a la siguiente es igual a \$200 multiplicado por el cuadrado de la diferencia de nivel. Es posible contar con niveles fraccionales gracias a que hay algunos empleados de tiempo parcial, y los datos de costos se aplican igual a estas fracciones.

Formulación. Si el análisis se basa en los datos disponibles, puede observarse que no vale la pena que ningún nivel de empleo sea mayor que 255, que corresponde a los requerimientos de la temporada pico. De esta manera, en primavera el número de empleados deberá ser de 255 y el problema se reduce a encontrar el nivel de las otras tres temporadas.

Para la formulación de programación dinámica, las temporadas deben ser las etapas. En realidad existe un número indefinido de etapas pues el problema se proyecta hacia un futuro indefinido.

Sin embargo, cada año comienza un ciclo idéntico y, como se conoce el nivel de empleados de la primavera, es factible tomar en cuenta sólo un ciclo de cuatro temporadas que termine en primavera, como se resume en seguida.

Etapas 1 = verano,

Etapas 2 = otoño,

Etapas 3 = invierno,

Etapas 4 = primavera.

x_n = nivel de empleados de la etapa n ($n = 1, 2, 3, 4$).

($x_4 = 255$.)

Es necesario que la temporada de primavera sea la última etapa porque debe conocerse o poderse obtener el valor óptimo de la variable de decisión de cada estado en esta última etapa, sin tener que considerar otras. En todas las demás etapas, la solución del nivel óptimo de empleo debe tomar en cuenta el efecto sobre los costos de la siguiente temporada.

Sea

r_n = mano de obra mínima que se requiere en la etapa n ,

donde estos requerimientos se dieron antes como $r_1 = 220$, $r_2 = 240$, $r_3 = 200$ y $r_4 = 255$. Así, los únicos valores factibles de x_n son

$$r_n \leq x_n \leq 255.$$

Si se hace referencia a los datos de costo que se proporcionaron en el enunciado del problema, se tiene

$$\text{Costo en la etapa } n = 200(x_n - x_{n-1})^2 + 2\,000(x_n - r_n).$$

Observe que el costo de la etapa actual depende sólo de la decisión x_n que se tome en esa etapa y del número de empleados de la etapa anterior x_{n-1} . Así, el nivel de empleados anterior es toda la información sobre el estado actual que se necesita para determinar la política óptima de ahí en adelante. Por ello, el estado s_n de la etapa n es

$$\text{Estado } s_n = x_{n-1}.$$

Cuando $n = 1$, $s_1 = x_0 = x_4 = 255$.

Para facilitar la referencia cuando se trabaje en el problema, en la tabla 10.3 se proporciona un resumen de los datos anteriores.

El objetivo en este problema es elegir a x_1, x_2, x_3 (con $x_0 = x_4 = 255$) para

$$\text{Minimizar} \quad \sum_{i=1}^4 [200(x_i - x_{i-1})^2 + 2\,000(x_i - r_i)],$$

sujeta a

$$r_i \leq x_i \leq 255, \quad \text{para } i = 1, 2, 3, 4.$$

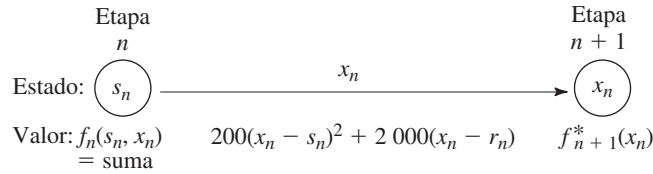
Entonces, de la etapa n en adelante ($n = 1, 2, 3, 4$), debido a que $s_n = x_{n-1}$

$$f_n(s_n, x_n) = 200(x_n - s_n)^2 + 2\,000(x_n - r_n) + \min_{r_i \leq x_i \leq 255} \sum_{i=n+1}^4 [200(x_i - x_{i-1})^2 + 2\,000(x_i - r_i)],$$

■ **TABLA 10.3** Datos del problema del local de Job Shop

n	r_n	Factible x_n	Posible $s_n = x_{n-1}$	Costo
1	220	$220 \leq x_1 \leq 255$	$s_1 = 255$	$200(x_1 - 255)^2 + 2\,000(x_1 - 220)$
2	240	$240 \leq x_2 \leq 255$	$220 \leq s_2 \leq 255$	$200(x_2 - x_1)^2 + 2\,000(x_2 - 240)$
3	200	$200 \leq x_3 \leq 255$	$240 \leq s_3 \leq 255$	$200(x_3 - x_2)^2 + 2\,000(x_3 - 200)$
4	255	$x_4 = 255$	$200 \leq s_4 \leq 255$	$200(255 - x_3)^2$

■ **FIGURA 10.8**
Estructura básica del problema del Local Job Shop.



donde esta sumatoria es igual a cero cuando $n = 4$ (porque no tiene términos). También,

$$f_n^*(s_n) = \min_{r_n \leq x_n \leq 255} f_n(s_n, x_n).$$

Entonces,

$$f_n(s_n, x_n) = 200(x_n - s_n)^2 + 2\,000(x_n - r_n) + f_{n+1}^*(x_n)$$

con f_5^* definida como cero, pues los costos después de la etapa 4 son irrelevantes para el análisis. En la figura 10.8 se presenta un resumen de estas relaciones básicas.

En consecuencia, la relación recursiva entre las funciones f_n^* es

$$f_n^*(s_n) = \min_{r_n \leq x_n \leq 255} \{200(x_n - s_n)^2 + 2\,000(x_n - r_n) + f_{n+1}^*(x_n)\}.$$

La programación dinámica utiliza esta relación para identificar en forma sucesiva estas funciones — $f_4^*(s_4), f_3^*(s_3), f_2^*(s_2), f_1^*(255)$ — y la x_n correspondiente que minimiza.

Procedimiento de solución. *Etapas 4:* Si se inicia en la última etapa ($n = 4$), se sabe que $x_4^* = 255$, de manera que los resultados son:

$n = 4:$	s_4	$f_4^*(s_4)$	x_4^*
	$200 \leq s_4 \leq 255$	$200(255 - s_4)^2$	255

Etapas 3: En el caso del problema que consiste nada más en las *dos* últimas etapas ($n = 3$), la relación recursiva se reduce a

$$\begin{aligned} f_3^*(s_3) &= \min_{200 \leq x_3 \leq 255} \{200(x_3 - s_3)^2 + 2\,000(x_3 - 200) + f_4^*(x_3)\} \\ &= \min_{200 \leq x_3 \leq 255} \{200(x_3 - s_3)^2 + 2\,000(x_3 - 200) + 200(255 - x_3)^2\}, \end{aligned}$$

donde los valores posibles de s_3 son $240 \leq s_3 \leq 255$.

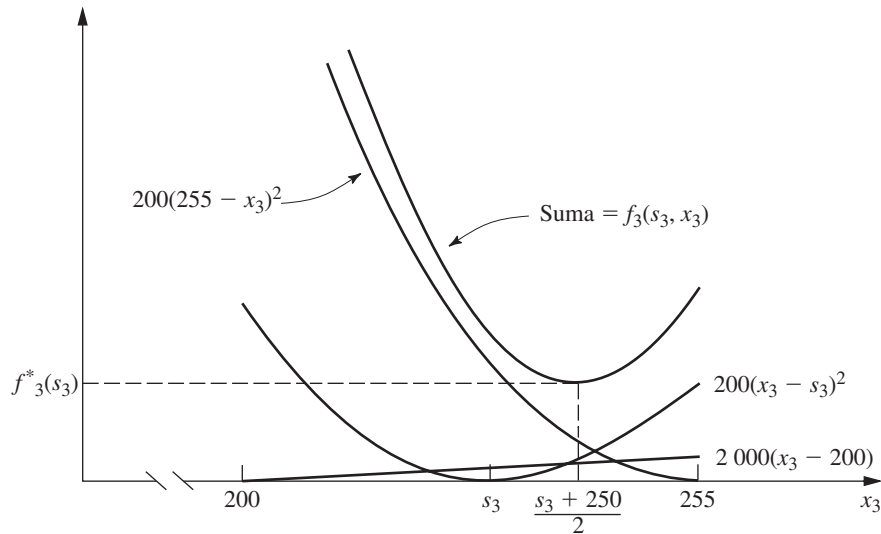
Una manera de obtener el valor de x_3 que minimiza $f_3(s_3, x_3)$ para cualquier valor dado de s_3 es el método gráfico que se muestra en la figura 10.9.

Sin embargo, una forma más rápida es usar *cálculo*. Se quiere obtener el valor de x_3 que minimiza, en términos de s_3 donde s_3 tiene un valor fijo (aunque desconocido). Se iguala a cero la primera derivada (parcial) de $f_3(s_3, x_3)$ con respecto a x_3 ,

$$\begin{aligned} \frac{\partial}{\partial x_3} f_3(s_3, x_3) &= 400(x_3 - s_3) + 2\,000 - 400(255 - x_3) \\ &= 400(2x_3 - s_3 - 250) \\ &= 0, \end{aligned}$$

lo que conduce a

$$x_3^* = \frac{s_3 + 250}{2}.$$



■ FIGURA 10.9

Solución gráfica de $f_3^*(s_3)$ del problema del Local Job Shop.

Como la segunda derivada es positiva y como esta solución se encuentra en el intervalo factible de x_3 ($200 \leq x_3 \leq 255$), para todos los valores posibles de s_3 ($240 \leq s_3 \leq 255$), sin duda se trata del mínimo que se busca.

Observe una diferencia clave entre la naturaleza de esta solución y la que se obtuvo en los ejemplos anteriores en donde sólo había que considerar unos cuantos estados posibles. Ahora se tiene un número *infinito* de estados posibles ($240 \leq s_3 \leq 255$), por lo que ya no es factible obtener por separado el valor de x_3^* para cada valor de s_3 . En lugar de ello, se obtiene x_3^* como una *función* de la incógnita s_3 .

Al utilizar

$$f_3^*(s_3) = f_3(s_3, x_3^*) = 200\left(\frac{s_3 + 250}{2} - s_3\right)^2 + 200\left(255 - \frac{s_3 + 250}{2}\right)^2 + 2\,000\left(\frac{s_3 + 250}{2} - 200\right)$$

y reducir de manera algebraica esta expresión, se obtienen los resultados que se requieren para el problema de la tercera etapa, que se resumen de la siguiente manera:

$n = 3$:	s_3	$f_3^*(s_3)$	x_3^*
	$240 \leq s_3 \leq 255$	$50(250 - s_3)^2 + 50(260 - s_3)^2 + 1\,000(s_3 - 150)$	$\frac{s_3 + 250}{2}$

Etapla 2: Los problemas de la segunda etapa ($n = 2$) y la primera etapa ($n = 1$) se resuelven en forma parecida. Para $n = 2$,

$$\begin{aligned} f_2(s_2, x_2) &= 200(x_2 - s_2)^2 + 2\,000(x_2 - r_2) + f_3^*(x_2) \\ &= 200(x_2 - s_2)^2 + 2\,000(x_2 - 240) \\ &\quad + 50(250 - x_2)^2 + 50(260 - x_2)^2 + 1\,000(x_2 - 150). \end{aligned}$$

Los valores posibles de s_2 son $220 \leq s_2 \leq 255$ y la región factible de x_2 es $240 \leq x_2 \leq 255$. El problema es minimizar el valor de x_2 en este intervalo, de modo que

$$f_2^*(s_2) = \min_{240 \leq x_2 \leq 255} f_2(s_2, x_2).$$

Si se iguala a cero la derivada parcial respecto de x_2 :

$$\begin{aligned}\frac{\partial}{\partial x_2} f_2(s_2, x_2) &= 400(x_2 - s_2) + 2\,000 - 100(250 - x_2) - 100(260 - x_2) + 1\,000 \\ &= 200(3x_2 - 2s_2 - 240) \\ &= 0\end{aligned}$$

se obtiene

$$x_2 = \frac{2s_2 + 240}{3}.$$

Puesto que

$$\frac{\partial^2}{\partial x_2^2} f_2(s_2, x_2) = 600 > 0,$$

este valor de x_2 sería el valor que da el mínimo *si fuera factible* ($240 \leq x_2 \leq 255$). En el caso de los valores posibles de s_2 ($220 \leq s_2 \leq 255$), de hecho esta solución es factible sólo si $240 \leq s_2 \leq 255$.

En consecuencia, todavía es necesario obtener el valor factible de x_2 que minimiza $f_2(s_2, x_2)$ cuando $220 \leq s_2 < 240$. La clave para analizar el comportamiento de $f_2(s_2, x_2)$ dentro de la región factible de x_2 es nuevamente la derivada parcial de $f_2(s_2, x_2)$. Cuando $s_2 < 240$,

$$\frac{\partial}{\partial x_2} f_2(s_2, x_2) > 0, \quad \text{para } 240 \leq x_2 \leq 255,$$

de manera que $x_2 = 240$ es el valor deseado que minimiza.

El siguiente paso es sustituir estos valores de x_2 en $f_2(s_2, x_2)$ para obtener $f_2^*(s_2)$ para $s_2 \geq 240$ y $s_2 < 240$. Con esto se obtiene

$n = 2:$	s_2	$f_2^*(s_2)$	x_2^*
	$220 \leq s_2 \leq 240$	$200(240 - s_2)^2 + 115\,000$	240
	$240 \leq s_2 \leq 255$	$\frac{200}{9} [(240 - s_2)^2 + (255 - s_2)^2 + (270 - s_2)^2] + 2\,000(s_2 - 195)$	$\frac{2s_2 + 240}{3}$

Etapla 1: En el problema de la primera etapa ($n = 1$),

$$f_1(s_1, x_1) = 200(x_1 - s_1)^2 + 2\,000(x_1 - r_1) + f_2^*(x_1).$$

Como $r_1 = 220$, la región factible para x_1 es $220 \leq x_1 \leq 255$. La expresión para $f_2^*(x_1)$ es diferente en los dos intervalos $220 \leq x_1 \leq 240$ y $240 \leq x_1 \leq 255$ de esta región. Por tanto,

$$f_1(s_1, x_1) = \begin{cases} 200(x_1 - s_1)^2 + 2\,000(x_1 - 220) + 200(240 - x_1)^2 + 115\,000, & \text{si } 220 \leq x_1 \leq 240 \\ 200(x_1 - s_1)^2 + 2\,000(x_1 - 220) + \frac{200}{9} [(240 - x_1)^2 + (255 - x_1)^2 + (270 - x_1)^2] \\ \quad + 2\,000(x_1 - 195), & \text{si } 240 \leq x_1 \leq 255. \end{cases}$$

Si se considera primero el caso en donde $220 \leq x_1 \leq 240$ se tiene

$$\begin{aligned}\frac{\partial}{\partial x_1} f_1(s_1, x_1) &= 400(x_1 - s_1) + 2\,000 - 400(240 - x_1) \\ &= 400(2x_1 - s_1 - 235).\end{aligned}$$

Se sabe que $s_1 = 255$ (nivel de empleados en primavera), por lo que

$$\frac{\partial}{\partial x_1} f_1(s_1, x_1) = 800(x_1 - 245) < 0$$

para toda $x_1 \leq 240$. Entonces, $x_1 = 240$ es el valor que minimiza a $f_1(s_1, x_1)$ en la región $220 \leq x_1 \leq 240$.

Cuando $240 \leq x_1 \leq 255$,

$$\begin{aligned} \frac{\partial}{\partial x_1} f_1(s_1, x_1) &= 400(x_1 - s_1) + 2\,000 \\ &\quad - \frac{400}{9}[(240 - x_1) + (255 - x_1) + (270 - x_1)] + 2\,000 \\ &= \frac{400}{3}(4x_1 - 3s_1 - 225). \end{aligned}$$

Como

$$\frac{\partial^2}{\partial x_1^2} f_1(s_1, x_1) > 0 \quad \text{para toda } x_1,$$

se establece

$$\frac{\partial}{\partial x_1} f_1(s_1, x_1) = 0,$$

lo que conduce a

$$x_1 = \frac{3s_1 + 225}{4}.$$

Como $s_1 = 255$, se concluye que $x_1 = 247.5$ minimiza el valor de $f_1(s_1, x_1)$ en la región $240 \leq x_1 \leq 255$.

Observe que esta región ($240 \leq x_1 \leq 255$) incluye $x_1 = 240$, de manera que $f_1(s_1, 240) > f_1(s_1, 247.5)$. En el penúltimo párrafo se encontró que $x_1 = 240$ minimiza $f_1(s_1, x_1)$ en la región $220 \leq x_1 \leq 240$. En consecuencia, se puede concluir que $x_1 = 247.5$ también minimiza $f_1(s_1, x_1)$ en toda la región factible $220 \leq x_1 \leq 255$.

El último cálculo se hace para encontrar $f_1^*(s_1)$ para $s_1 = 255$ al sustituir $x_1 = 247.5$ en la expresión $f_1(255, x_1)$ que se cumple para $240 \leq x_1 \leq 255$. Entonces,

$$\begin{aligned} f_1^*(255) &= 200(247.5 - 255)^2 + 2\,000(247.5 - 220) \\ &\quad + \frac{200}{9} [2(250 - 247.5)^2 + (265 - 247.5)^2 + 30(742.5 - 575)] \\ &= 185\,000. \end{aligned}$$

Estos resultados se resumen de la siguiente manera:

$n = 1$:	s_1	$f_1^*(s_1)$	x_1^*
	255	185 000	247.5

Por tanto, al regresar a través de las tablas que se obtuvieron para $n = 2$, $n = 3$ y $n = 4$, respectivamente, y establecer cada vez $s_n = x_{n-1}^*$, la solución óptima que se obtiene es $x_1^* = 247.5$, $x_2^* = 245$, $x_3^* = 247.5$, $x_4^* = 255$, con un costo estimado total por ciclo de \$185 000.

Para concluir la ejemplificación de programación dinámica determinística se analizará un problema que requiere *más de una* variable para describir el estado en cada etapa.

EJEMPLO 5 Problema de la Wyndor Glass Company

Considere el siguiente problema de programación lineal:

$$\text{Maximizar } Z = 3x_1 + 5x_2,$$

sujeta a

$$x_1 \leq 4$$

$$2x_2 \leq 12$$

$$3x_1 + 2x_2 \leq 18$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

(Puede reconocerse que este modelo es el que se estableció para manejar el problema de la Wyndor Glass Co., en la sección 3.1.) Una manera de resolver problemas pequeños de programación lineal —o no lineal— como éste es emplear programación dinámica, aspecto que se ilustra a continuación.

Formulación. Este problema requiere tomar dos decisiones interrelacionadas, es decir, el nivel de la actividad 1, denotado por x_1 , y el nivel de la actividad 2, denotado por x_2 . Por ello, estas dos actividades se pueden interpretar como las dos etapas de una formulación de programación dinámica. Aunque es posible tomarlas en cualquier orden, sea la etapa n = la actividad n ($n = 1, 2$). Así, x_n es la variable de decisión en la etapa n .

¿Cuáles son los estados? Es decir, dado que se toma una decisión en las etapas anteriores (si las hay), ¿qué información se necesita conocer sobre el estado actual de las cosas antes de poder tomar una decisión en la etapa n ? Un poco de reflexión puede sugerir que la información requerida es la *cantidad de holgura* que queda en las restricciones funcionales. El lado derecho de estas restricciones (4, 12 y 18) se interpreta como la cantidad total disponible de los recursos 1, 2 y 3, respectivamente (como se describió en la sección 3.1). Entonces el estado s_n se puede definir como

Estado s_n = cantidad de los respectivos recursos todavía disponibles para ser asignados a las actividades restantes.

(Observe que esta definición de estado es análoga a la de los problemas de distribución de esfuerzo que incluyen los ejemplos 2 y 3, excepto que ahora se tienen tres tipos de recursos en lugar de uno.) Entonces,

$$s_n = (R_1, R_2, R_3),$$

donde R_i es la cantidad disponible del recurso i ($i = 1, 2, 3$). Por tanto,

$$s_1 = (4, 12, 18),$$

$$s_2 = (4 - x_1, 12, 18 - 3x_1).$$

Sin embargo, al comenzar este problema en la etapa 2, todavía no se conoce el valor de x_1 , por lo que en ese momento se usa $s_2 = (R_1, R_2, R_3)$.

Al contrario de los ejemplos anteriores, este problema tiene *tres* variables de estado —es decir, un *vector de estado* con tres componentes— en lugar de una en cada etapa. Desde un punto de vista teórico, esta diferencia no es seria. Sólo significa que, en lugar de considerar todos los valores posibles de una variable de estado, deben considerarse todas las *combinaciones* posibles de valores de varias variables de estado. Sin embargo, desde un punto de vista de eficiencia de los cálculos, esta diferencia tiende a ser una complicación muy seria. Como en general el número de combinaciones puede ser tan grande como el *producto* del número de valores posibles de las respectivas variables, la cantidad de cálculos que se requieren tiende a “explotar” con rapidez cuando se introduce variables de estado adicionales. Este fenómeno ha recibido el adecuado nombre de la **calamidad de la dimensión**.

Cada una de las tres variables de estado es *continua*, por lo que, en lugar de considerar por separado las combinaciones posibles de valores, debe emplearse el método que se introdujo en el ejemplo 4 para obtener la información requerida como una *función* del estado del sistema.

Pese a estas complicaciones, el problema es lo suficientemente pequeño como para poder resolverlo sin mucha dificultad. Para hacerlo es necesario introducir la notación usual de programación dinámica. Así,

$$f_2(R_1, R_2, R_3, x_2) = \text{contribución de la actividad 2 a } Z \text{ si el sistema se encuentra en el estado } (R_1, R_2, R_3) \text{ al iniciar la etapa 2 y la decisión es } x_2 \\ = 5x_2,$$

$$f_1(4, 12, 18, x_1) = \text{contribución de las actividades 1 y 2 a } Z \text{ si el sistema se encuentra en el estado } (4, 12, 18) \text{ al iniciar la etapa 1, la decisión inmediata es } x_1 \text{ y después se toma una decisión óptima en la etapa 2.} \\ = 3x_1 + \max_{\substack{2x_2 \leq 12 \\ 2x_2 \leq 18 - 3x_1 \\ x_2 \geq 0}} \{5x_2\}.$$

De manera similar, para $n = 1, 2$,

$$f_n^*(R_1, R_2, R_3) = \max_{x_n} f_n(R_1, R_2, R_3, x_n),$$

donde este máximo se toma sobre los valores factibles de x_n , por lo cual, al usar la parte relevante de las restricciones del problema, se obtiene

$$(1) \quad f_2^*(R_1, R_2, R_3) = \max_{\substack{2x_2 \leq R_2 \\ 2x_2 \leq R_3 \\ x_2 \geq 0}} \{5x_2\},$$

$$(2) \quad f_1(4, 12, 18, x_1) = 3x_1 + f_2^*(4 - x_1, 12, 18 - 3x_1),$$

$$(3) \quad f_1^*(4, 12, 18) = \max_{\substack{x_1 \leq 4 \\ 3x_1 \leq 18 \\ x_1 \geq 0}} \{3x_1 + f_2^*(4 - x_1, 12, 18 - 3x_1)\}.$$

La ecuación (1) se usará para resolver el problema de 2 etapas. La ecuación (2) muestra la estructura básica de programación dinámica para este problema, como también se puede ver en la figura 10.10. La ecuación (3) proporciona una *relación recursiva* entre f_1^* y f_2^* que se usará para resolver el problema de la etapa 1.

Procedimiento de solución. *Etapla 2:* Para resolver la última etapa ($n = 2$), la ecuación (1) indica que x_2^* debe ser el valor más grande de x_2 que *satisface de manera simultánea* $2x_2 \leq R_2$, $2x_2 \leq R_3$ y $x_2 \geq 0$. Si se supone que $R_2 \geq 0$ y $R_3 \geq 0$, para que existan soluciones factibles, este valor más grande es el más pequeño de $R_2/2$ y $R_3/2$. Por consiguiente, la solución es

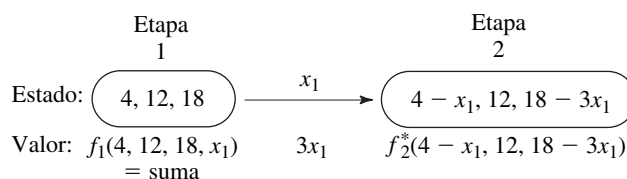
$n = 2:$	(R_1, R_2, R_3)	$f_2^*(R_1, R_2, R_3)$	x_2^*
	$R_2 \geq 0, R_3 \geq 0$	$5 \min \left\{ \frac{R_2}{2}, \frac{R_3}{2} \right\}$	$\min \left\{ \frac{R_2}{2}, \frac{R_3}{2} \right\}$

Etapla 1: Para resolver el problema de dos etapas ($n = 1$), se sustituye la solución que se acaba de obtener para $f_2^*(R_1, R_2, R_3)$ en la ecuación (3). En la etapa 2,

$$(R_1, R_2, R_3) = (4 - x_1, 12, 18 - 3x_1),$$

■ FIGURA 10.10

Estructura básica del problema de programación lineal de la Wyndor Glass Co.



de manera que

$$f_2^*(4 - x_1, 12, 18 - 3x_1) = 5 \min \left\{ \frac{R_2}{2}, \frac{R_3}{2} \right\} = 5 \min \left\{ \frac{12}{2}, \frac{18 - 3x_1}{2} \right\}$$

es la solución específica que se tiene que sustituir en la ecuación (3). Después de combinar las restricciones sobre x_1 , esta ecuación se convierte en

$$f_1^*(4, 12, 18) = \max_{0 \leq x_1 \leq 4} \left\{ 3x_1 + 5 \min \left\{ \frac{12}{2}, \frac{18 - 3x_1}{2} \right\} \right\}.$$

Observe que en el intervalo factible $0 \leq x_1 \leq 4$,

$$\min \left\{ \frac{12}{2}, \frac{18 - 3x_1}{2} \right\} = \begin{cases} 6 & \text{si } 0 \leq x_1 \leq 2 \\ 9 - \frac{3}{2}x_1 & \text{si } 2 \leq x_1 \leq 4, \end{cases}$$

de manera que

$$3x_1 + 5 \min \left\{ \frac{12}{2}, \frac{18 - 3x_1}{2} \right\} = \begin{cases} 3x_1 + 30 & \text{si } 0 \leq x_1 \leq 2 \\ 45 - \frac{9}{2}x_1 & \text{si } 2 \leq x_1 \leq 4 \end{cases}$$

Como las dos funciones

$$\max_{0 \leq x_1 \leq 2} \{3x_1 + 30\} \quad \text{y} \quad \max_{2 \leq x_1 \leq 4} \left\{ 45 - \frac{9}{2}x_1 \right\}$$

adquieren su máximo en $x_1 = 2$, se concluye que $x_1^* = 2$ y que este máximo es 36, como se expresa en la siguiente tabla.

$n = 1:$	(R_1, R_2, R_3)	$f_1^*(R_1, R_2, R_3)$	x_1^*
	(4, 12, 18)	36	2

Como el valor $x_1^* = 2$ conduce a

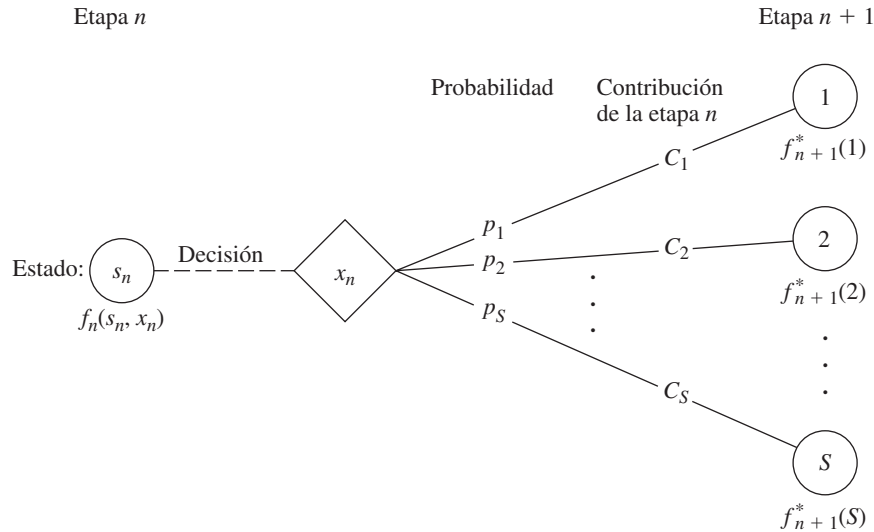
$$R_1 = 4 - 2 = 2, \quad R_2 = 12, \quad R_3 = 18 - 3(2) = 12$$

para la etapa 2, la tabla de $n = 2$ lleva a $x_2^* = 6$. En consecuencia, $x_1^* = 2, x_2^* = 6$ es la solución óptima para este problema, lo que coincide con la solución original que se encontró en la sección 3.1; la tabla de $n = 1$ muestra que el valor de Z que resulta es 36.

Hasta el momento, usted ha visto una gran variedad de aplicaciones de la programación dinámica y en la sección siguiente estudiará todavía más. Sin embargo, estos ejemplos sólo tocan el tema de manera superficial. Por ejemplo, en el capítulo 2, la referencia seleccionada 4 describe 47 tipos de problemas en los que se puede aplicar la programación dinámica. (Esta referencia también presenta una herramienta de software que puede utilizarse para resolver estos tipos de problemas.) El punto en común que aparece en estas aplicaciones de programación dinámica es la necesidad de tomar una serie de decisiones interrelacionadas y la forma eficiente que proporciona la programación dinámica para encontrar una combinación de decisiones óptima.

10.4 PROGRAMACIÓN DINÁMICA PROBABILÍSTICA

La programación dinámica *probabilística* difiere de la determinística en que el estado de la siguiente etapa *no* está determinado por completo por el estado y la política de decisión de la etapa actual. En su lugar, existe una *distribución de probabilidad* para determinar cuál será el siguiente estado. Sin embargo, esta distribución de probabilidad queda completamente determinada por el estado y



■ **FIGURA 10.11**
Estructura básica de
programación dinámica
probabilística.

la política de decisión de la etapa actual. En el diagrama de la figura 10.11 se describe la estructura básica que resulta para los problemas de programación dinámica probabilística.

En lo que se refiere a este diagrama, sea S el número de estados posibles en la etapa $n+1$ y etiquete estos estados en el lado derecho con $1, 2, \dots, S$. El sistema cambia al estado i con probabilidad p_i ($i = 1, 2, \dots, S$) dados el estado s_n y la decisión x_n en la etapa n . Si el sistema cambia al estado i , C_i es la contribución de la etapa n a la función objetivo.

Cuando se expande la figura 10.11 para incluir todos los estados y las decisiones posibles en todas las etapas, se obtiene lo que con frecuencia se conoce como **árbol de decisión**, el cual, si no es muy grande, proporciona una forma útil de resumir las distintas posibilidades.

Debido a la estructura probabilística, la relación entre $f_n(s_n, x_n)$ y $f_{n+1}^*(s_{n+1})$ necesariamente es más complicada que en el caso determinístico. La forma exacta de esta relación dependerá de la forma global de la función objetivo.

Para ilustrar lo anterior, suponga que el objetivo es *minimizar la suma esperada* de las contribuciones de las etapas individuales. En este caso, $f_n(s_n, x_n)$ representa la suma esperada mínima de la etapa n en adelante, *dado* que en la etapa n el estado es s_n y la política de decisión es x_n . En consecuencia,

$$f_n(s_n, x_n) = \sum_{i=1}^S p_i [C_i + f_{n+1}^*(i)],$$

con

$$f_{n+1}^*(i) = \min_{x_{n+1}} f_{n+1}(i, x_{n+1}),$$

donde la minimización se toma sobre todos los valores *factibles* de x_{n+1} .

El ejemplo 6 tiene esta misma forma. El ejemplo 7 ilustrará una distinta.

EJEMPLO 6 Determinación de holguras por rechazos

La HIT-AND-MISS MANUFACTURING COMPANY ha recibido un pedido para surtir un artículo de tipo especial. El cliente ha especificado estándares de calidad tan rigurosos que es posible que el fabricante tenga que producir más de un artículo para obtener uno aceptable. El número *adicional* de artículos que se producen en una corrida de producción se llama *holgura por rechazos*. Es una práctica común incluir una holgura por rechazos cuando se produce el pedido de un cliente y, en este caso, parece conveniente.

El fabricante estima que cada unidad producida de este tipo tiene una probabilidad de $\frac{1}{2}$ de ser *aceptable* y una probabilidad de $\frac{1}{2}$ de ser *defectuosa* (sin posibilidad de corrección). En consecuen-

cia, el número de unidades aceptables producidas en un lote de tamaño L tendrá una *distribución binomial*, es decir, la probabilidad de producir cero artículos aceptables en ese lote es $(\frac{1}{2})^L$.

Los costos marginales de producción se estiman en \$100 por unidad (aunque sea defectuosa), y los artículos adicionales se desperdician. Además, se incurre en costos fijos de \$300 cada vez que se pone en marcha el proceso de producción para elaborar este artículo y se requiere una preparación completa a este mismo costo para cada corrida de producción subsecuente si el procedimiento de inspección revela que en todo un lote no hubo una unidad aceptable. El fabricante tiene tiempo para realizar hasta tres corridas de producción. Si al final de la tercera corrida no obtiene un artículo aceptable, el costo ocasionado por la venta perdida y las multas será de \$1 600.

El objetivo es determinar la política sobre el tamaño del lote ($1 +$ holgura por rechazos) de las corridas de producción que se requieren, de tal modo que se minimice el costo total esperado por el fabricante.

Formulación. La formulación de este problema de programación dinámica es:

Etapla $n = n$ -ésima corrida de producción ($n = 1, 2, 3$),

$x_n =$ tamaño del lote de la etapa n ,

Etapla $s_n =$ número de productos aceptables por obtener (1 o 0) al iniciar la etapa n .

Así, el estado es $s_1 = 1$ en la etapa 1. Si en lo subsecuente se obtiene al menos un artículo aceptable, el estado cambia a $s_n = 0$, después de lo cual no se incurre en costos adicionales.

Según el objetivo que se estableció para el problema,

$f_n(s_n, x_n) =$ costo total esperado en las etapas $n, \dots, 3$ si en la etapa n el sistema comienza en el estado s_n , la decisión inmediata es x_n , y en adelante se toman decisiones óptimas,

$$f_n^*(s_n) = \min_{x_n=0, 1, \dots} f_n(s_n, x_n),$$

donde $f_n^*(0) = 0$. Si se usa \$100 como unidad monetaria, la contribución al costo de la etapa n es $[K(x_n) + x_n]$ independientemente del siguiente estado, donde $K(x_n)$ es una función de x_n tal que

$$K(x_n) = \begin{cases} 0, & \text{si } x_n = 0 \\ 3, & \text{si } x_n > 0. \end{cases}$$

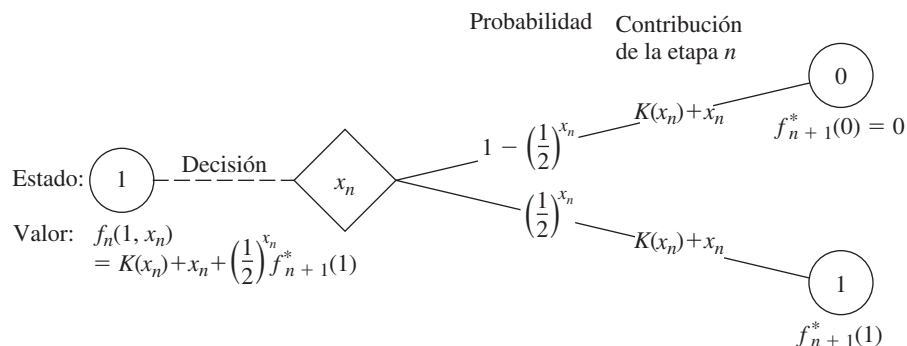
Por tanto, para $s_n = 1$,

$$\begin{aligned} f_n(1, x_n) &= K(x_n) + x_n + \left(\frac{1}{2}\right)^{x_n} f_{n+1}^*(1) + \left[1 - \left(\frac{1}{2}\right)^{x_n}\right] f_{n+1}^*(0) \\ &= K(x_n) + x_n + \left(\frac{1}{2}\right)^{x_n} f_{n+1}^*(1) \end{aligned}$$

[donde $f_4^*(1)$ es igual a 16, el costo terminal por no haber obtenido artículos aceptables]. En la figura 10.12 se proporciona un resumen de estas relaciones básicas.

■ FIGURA 10.12

Estructura básica del problema de programación lineal de la Hit-and-Miss Manufacturing Co.



En consecuencia, la relación recursiva de los cálculos de programación dinámica es

$$f_n^*(1) = \min_{x_n=0, 1, \dots} \left\{ K(x_n) + x_n + \left(\frac{1}{2}\right)^{x_n} f_{n+1}^*(1) \right\}$$

para $n = 1, 2, 3$.

Procedimiento de solución. Los cálculos que utilizan esta relación recursiva se resumen de la siguiente manera.

		$f_3(1, x_3) = K(x_3) + x_3 + 16\left(\frac{1}{2}\right)^{x_3}$						$f_3^*(s_3)$	x_3^*
		0	1	2	3	4	5		
$n = 3:$	s_3								
	0	0						0	0
	1	16	12	9	8	8	$8\frac{1}{2}$	8	3 o 4

		$f_2(1, x_2) = K(x_2) + x_2 + \left(\frac{1}{2}\right)^{x_2} f_3^*(1)$					$f_2^*(s_2)$	x_2^*
		0	1	2	3	4		
$n = 2:$	s_2							
	0	0					0	0
	1	8	8	7	7	$7\frac{1}{2}$	7	2 o 3

		$f_1(1, x_1) = K(x_1) + x_1 + \left(\frac{1}{2}\right)^{x_1} f_2^*(1)$					$f_1^*(s_1)$	x_1^*
		0	1	2	3	4		
$n = 1:$	s_1							
	1	7	$7\frac{1}{2}$	$6\frac{3}{4}$	$6\frac{7}{8}$	$7\frac{7}{16}$	$6\frac{3}{4}$	2

Por tanto, la política óptima es producir dos artículos en la primera corrida de producción; si ninguno fuera aceptable, entonces habría que producir dos o tres artículos en la segunda corrida; si ninguno resultara aceptable, entonces se producirían tres o cuatro artículos en la tercera corrida. El costo total esperado de esta política es de \$675.

EJEMPLO 7 Ganadora en Las Vegas

Una joven emprendedora experta en estadística cree haber desarrollado un sistema para ganar un popular juego en Las Vegas. Sus colegas no piensan que este sistema sea tan bueno, por lo que le apuestan que si comienza con tres fichas, ella no tendrá al menos cinco fichas después de tres jugadas. Cada jugada incluye apostar cualquier cantidad de las fichas disponibles y ganar o perder este mismo número de fichas. La joven cree que su sistema le dará una probabilidad de $\frac{2}{3}$ de ganar una jugada dada.

Si se supone que la experta en estadística está en lo correcto, se quiere usar programación dinámica para determinar su política óptima sobre cuántas fichas apostar (si apuesta) en cada una de las tres jugadas. La decisión en cada jugada deberá tomar en cuenta los resultados de las jugadas anteriores. El objetivo es maximizar la probabilidad de ganar la apuesta hecha a sus colegas.

Formulación. La formulación de programación dinámica para este problema es

Etapla $n = n$ -ésima jugada del juego ($n = 1, 2, 3$),

$x_n =$ número de fichas que debe apostar en la etapa n ,

Etapla $s_n =$ número de fichas disponibles para comenzar la etapa n .

Se eligió esta definición del estado del sistema porque proporciona la información necesaria sobre la situación actual para tomar una decisión óptima sobre cuántas fichas apostar en la siguiente jugada.

Como el objetivo es maximizar la probabilidad de que la joven gane la apuesta, la función objetivo que debe maximizarse en cada etapa es la probabilidad de terminar las tres jugadas con cinco fichas o más. (Observe que el valor de terminar con más de cinco fichas es el mismo que el de terminar con cinco fichas, ya que la apuesta se gana de las dos formas.) Así,

$f_n(s_n, x_n) =$ probabilidad de terminar las tres jugadas con cinco fichas o más, dado que la joven comienza la etapa n con s_n fichas, su decisión inmediata es x_n y en adelante toma decisiones óptimas,

$$f_n^*(s_n) = \max_{x_n=0, 1, \dots, s_n} f_n(s_n, x_n).$$

La expresión para $f_n(s_n, x_n)$ debe reflejar el hecho de que, en un momento dado, es posible acumular cinco fichas aun cuando pierda la siguiente jugada. Si pierde, el estado en la siguiente etapa será $s_n - x_n$, y la probabilidad de terminar con al menos cinco fichas será entonces $f_{n+1}^*(s_n - x_n)$. Por el contrario, si gana la siguiente jugada, el estado del sistema será $s_n + x_n$ y la probabilidad correspondiente será $f_{n+1}^*(s_n + x_n)$. Como se supone que la probabilidad de ganar una jugada dada es $\frac{2}{3}$, se concluye que

$$f_n(s_n, x_n) = \frac{1}{3} f_{n+1}^*(s_n - x_n) + \frac{2}{3} f_{n+1}^*(s_n + x_n)$$

[donde $f_4^*(s_4)$ se define como 0 para $s_4 < 5$ y 1 para $s_4 \geq 5$]. Entonces, no existe una contribución directa a la función objetivo por parte de la etapa n además del efecto que tiene estar en el siguiente estado. En la figura 10.13 se resumen estas relaciones básicas.

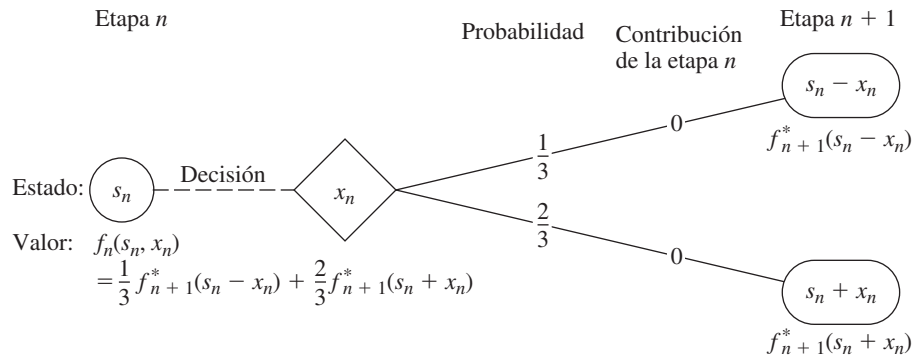
Entonces, la relación recursiva para este problema es

$$f_n^*(s_n) = \max_{x_n=0, 1, \dots, s_n} \left\{ \frac{1}{3} f_{n+1}^*(s_n - x_n) + \frac{2}{3} f_{n+1}^*(s_n + x_n) \right\},$$

para $n = 1, 2, 3$, con $f_4^*(s_4)$ como se definió.

Procedimiento de solución. Esta relación recursiva conduce a los siguientes resultados de cálculos.

■ **FIGURA 10.13**
Estructura básica del problema de Las Vegas.



$n = 3$:		s_3	$f_3^*(s_3)$	x_3^*
	0	0	—	—
	1	0	—	—
	2	0	—	—
	3	$\frac{2}{3}$	2 (o más)	
	4	$\frac{2}{3}$	1 (o más)	
	≥ 5	1	0 ($0 \leq s_3 - 5$)	

		$f_2(s_2, x_2) = \frac{1}{3}f_3^*(s_2 - x_2) + \frac{2}{3}f_3^*(s_2 + x_2)$						
$n = 2$:	s_2	x_2					$f_2^*(s_2)$	x_2^*
		0	1	2	3	4		
	0	0					0	—
	1	0	0				0	—
	2	0	$\frac{4}{9}$	$\frac{4}{9}$			$\frac{4}{9}$	1 o 2
	3	$\frac{2}{3}$	$\frac{4}{9}$	$\frac{2}{3}$	$\frac{2}{3}$		$\frac{2}{3}$	0, 2 o 3
	4	$\frac{2}{3}$	$\frac{8}{9}$	$\frac{2}{3}$	$\frac{2}{3}$	$\frac{2}{3}$	$\frac{8}{9}$	1
	≥ 5	1					1	0 ($0 \leq s_2 - 5$)

		$f_1(s_1, x_1) = \frac{1}{3}f_2^*(s_1 - x_1) + \frac{2}{3}f_2^*(s_1 + x_1)$					
$n = 1$:	s_1	x_1				$f_1^*(s_1)$	x_1^*
		0	1	2	3		
	3	$\frac{2}{3}$	$\frac{20}{27}$	$\frac{2}{3}$	$\frac{2}{3}$	$\frac{20}{27}$	1

Por tanto, la política óptima es

$$x_1^* = 1 \begin{cases} \text{si gana,} & x_2^* = 1 \begin{cases} \text{si gana,} & x_3^* = 0 \\ \text{si pierde,} & x_3^* = 2 \text{ o } 3. \end{cases} \\ \text{si pierde,} & x_2^* = 1 \text{ o } 2 \begin{cases} \text{si gana,} & x_3^* = \begin{cases} 2 \text{ o } 3 & (\text{para } x_2^* = 1) \\ 1, 2, 3 \text{ o } 4 & (\text{para } x_2^* = 2) \end{cases} \\ \text{si pierde,} & \text{la apuesta está perdida} \end{cases} \end{cases}$$

Esta política da a la experta en estadística una probabilidad de $\frac{20}{27}$ de ganar la apuesta a sus colegas.

10.5 CONCLUSIONES

La programación dinámica es una técnica muy útil para tomar una *sucesión de decisiones interrelacionadas*. Requiere la formulación de una *relación recursiva* apropiada para cada problema individual. Sin embargo, proporciona grandes ahorros computacionales en comparación con la enumeración exhaustiva para encontrar la mejor combinación de decisiones, en especial cuando se trata de problemas grandes. Por ejemplo, si un problema tiene 10 etapas con 10 estados y 10 decisiones posibles en cada etapa, la enumeración exhaustiva tendría que considerar hasta 10 mil millones de combinaciones, mientras que la programación dinámica necesita hacer como máximo mil cálculos (10 para cada estado en cada etapa).